西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验一 EMU8086 使用及 8086 指令系统

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 04 月 13 日

成 绩

指导教师评语:
指导教师: 年 月 日

一、 实验要求

1. 熟悉并掌握 EMU8086 汇编语言编程调试环境；
2. 学习 8086 寻址方式（立即寻址、直接寻址、寄存器寻址、寄存器间接寻址、寄存器相对寻址、基址变址寻址、基址变址相对寻址、隐含寻址）。要求对以上寻址方式进行熟悉，观察不同寻址方式下指令运行结果（每种寻址方式 2 个实例，可参考 PPT 及教材相关章节）；
3. 学习 8086 指令系统，输入简单的指令（程序），观察各寄存器、内存相关单元以及处理器标志位的变化（数据传送类指令，算术运算类指令，逻辑运算类指令，标志处理和 CPU 控制类指令，移位和循环移位类指令，处理器控制类指令、程序控制类指令、输入/输出类指令等，要求每类指令至少 2 个用例。具体用例自行设计，可参考 PPT 及教材用例）；
4. 学习汇编语言程序设计的基本步骤和方法；
5. 学会使用 EMU8086 debug 调试程序（单步运行、断点设置等）；
6. 学习汇编编程中 DOS 功能调用方法，掌握常用的 DOS 功能调用；
7. 编写一个简单的程序：将“This is my first ASM program-姓名（汉语拼音各人的姓名）”放在 DS=0700H，BX=0000H 开始的存储器单元中，然后将该内容搬移到 BX=0100H 开始的单元中，最后将该字符串通过 DOS 功能调用显示在屏幕上。

二、 实验目的

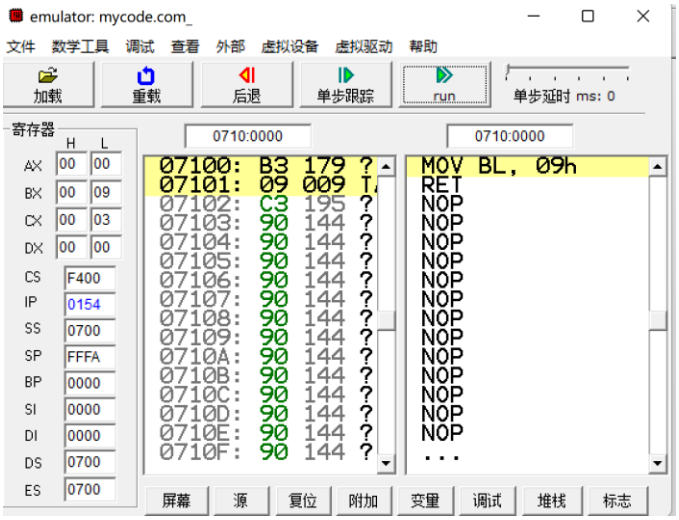
1. 学习 EMU8086 仿真开发环境的使用，理解和掌握汇编语言编程的基本步骤；
2. 熟悉并掌握 8086/8088 指令系统及内部结构；
3. 熟悉常用的 DOS 功能调用，掌握汇编语言编程的人机交互方法；
4. 熟悉变量、常量及伪指令的使用；
5. 熟悉内存单元的存储结构，字符串的处理以及简单的编程。

三、 实验代码及实验结果

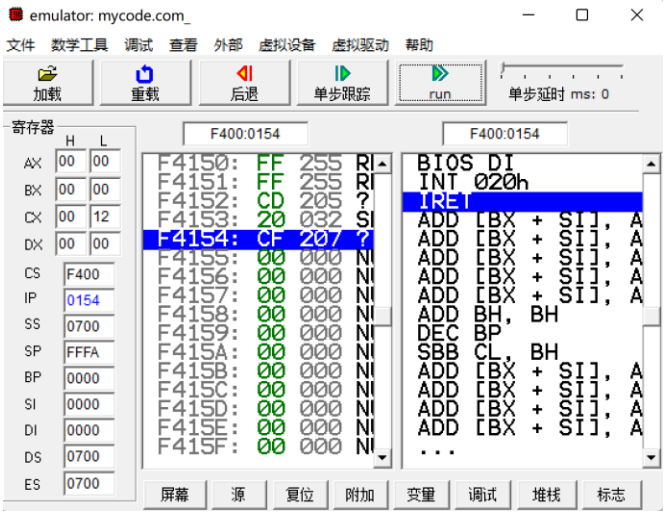
2. 寻址方式

1）立即寻址：

```
MOV BL, 09h
```



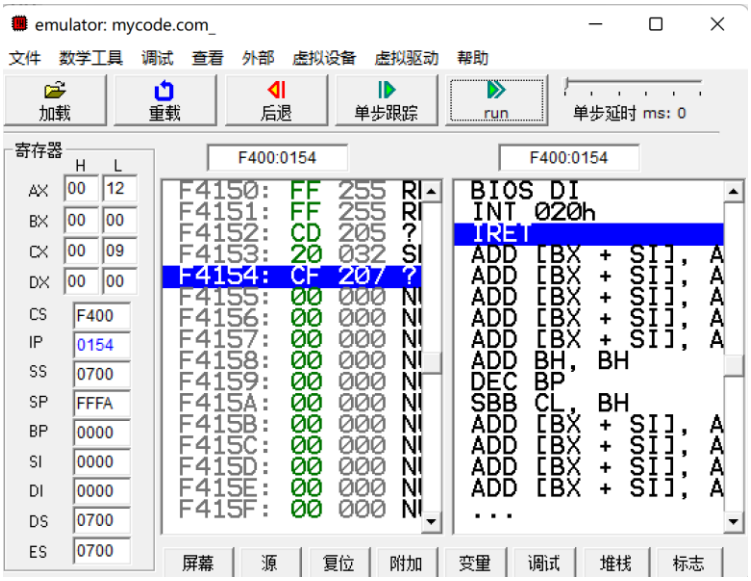
MOV CX, 12h



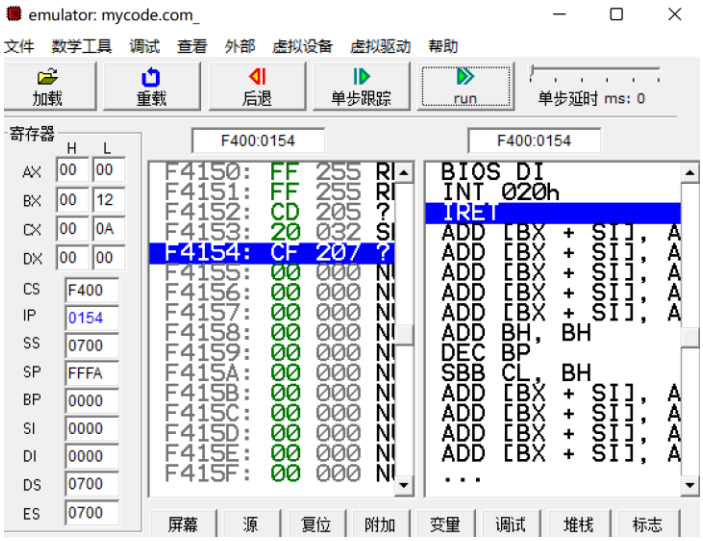
2）直接寻址：

MOV [08h], 12h

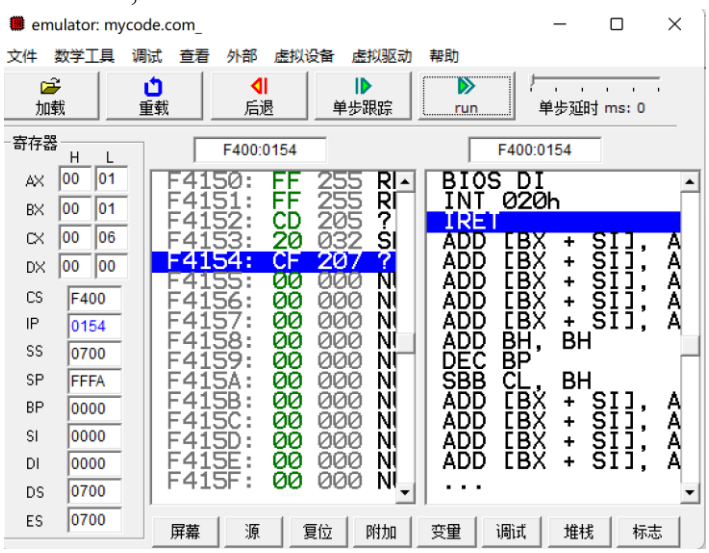
MOV AX, [08h]



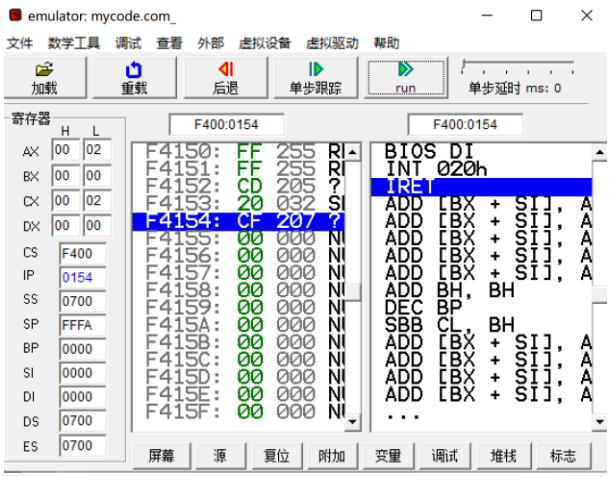
MOV [1200h], 12h
MOV BX, [1200h]



3）寄存器寻址：
MOV BX, 01h
MOV AX, BX



MOV CX, 02h
MOV AX, CX

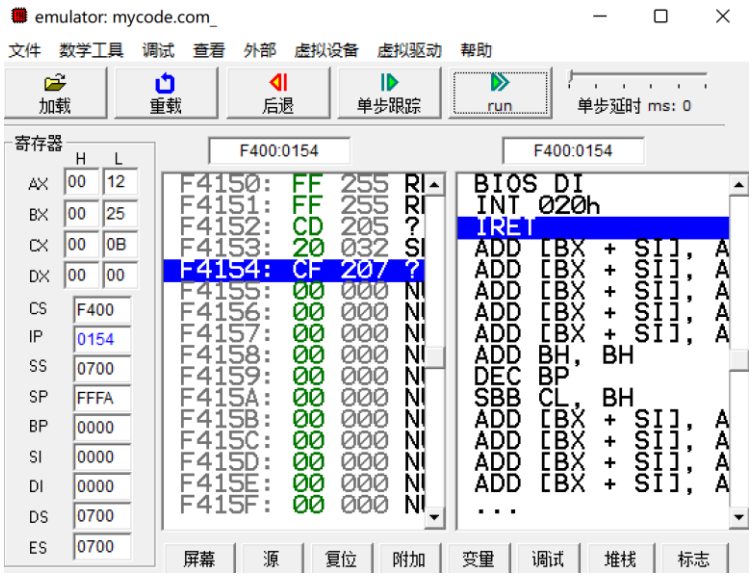


4）寄存器间接寻址：

MOV [25h], 12h

MOV BX, 25h

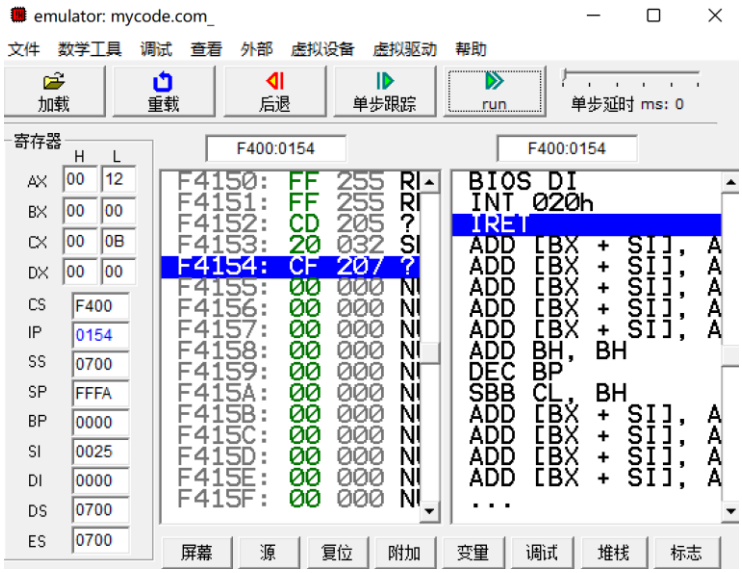
MOV AX, [BX]



MOV [25h], 12h

MOV SI, 25h

MOV AX, [SI]

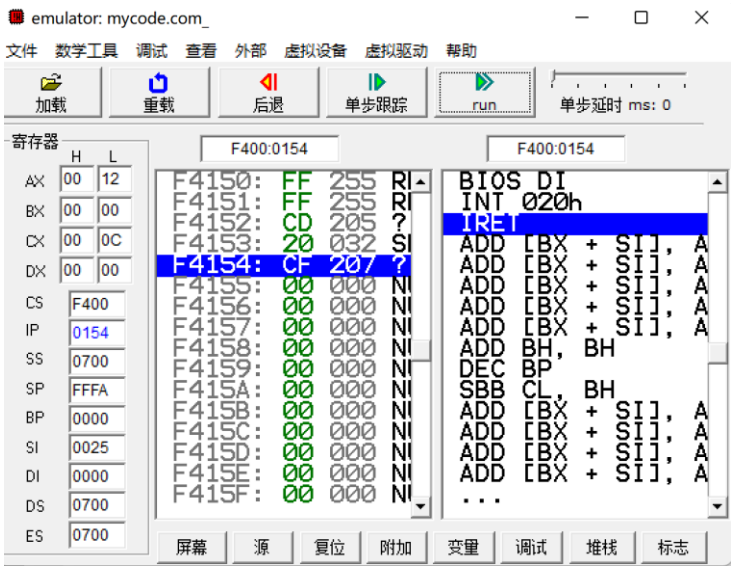


5）寄存器相对寻址：

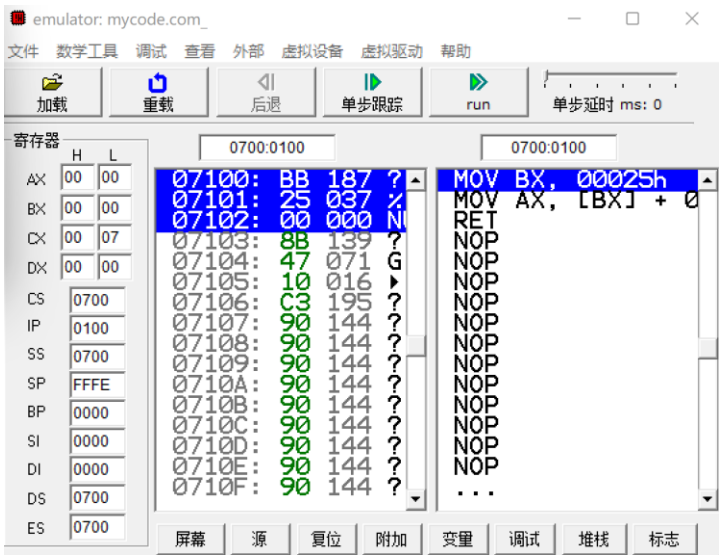
MOV [35h], 12h

MOV SI, 25h

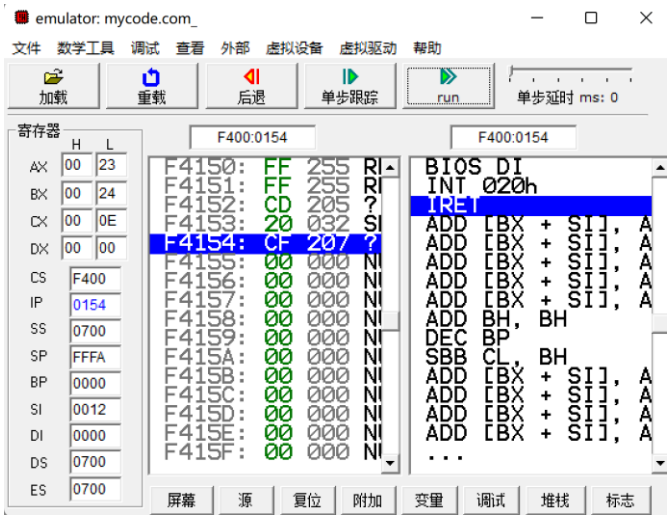
MOV AX, [SI+10h]



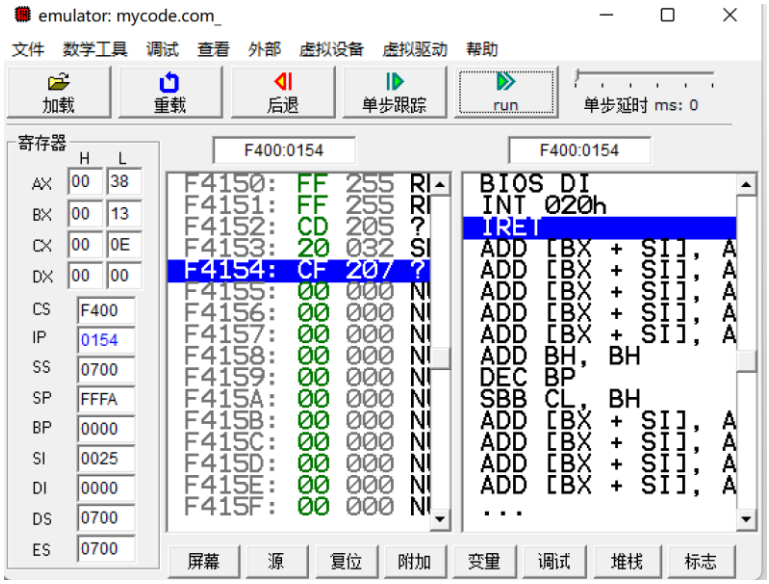
MOV [35h], 12h
MOV BX, 25h
MOV AX, [BX+10h]



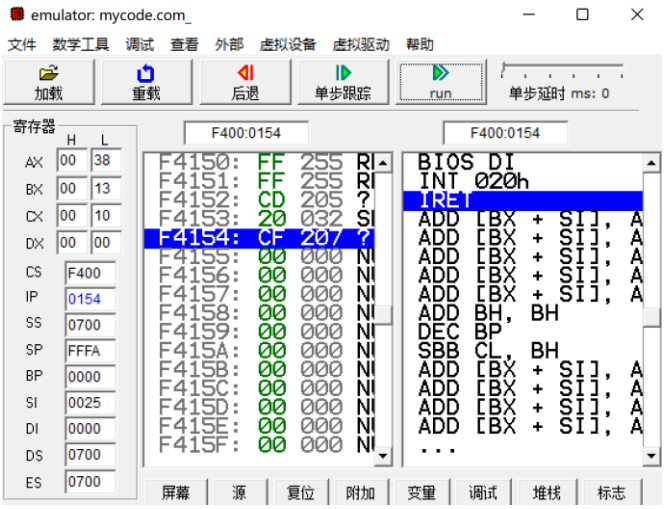
6）基址变址寻址：
MOV [36h], 23h
MOV SI, 12h
MOV BX, 24h
MOV AX, [BX][SI]



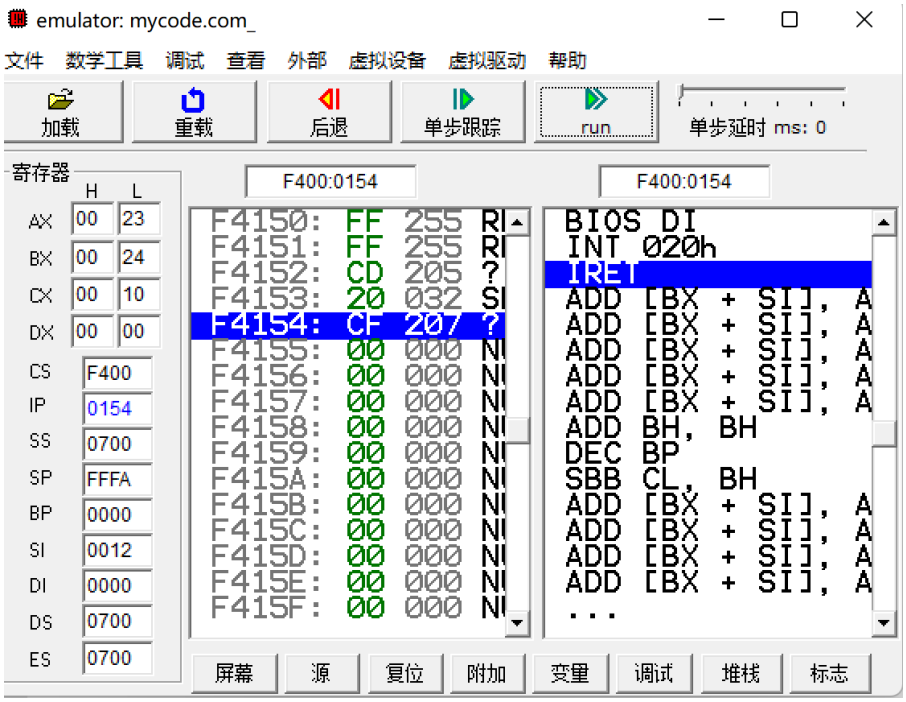
MOV [38h], 38h
MOV SI, 25h
MOV BX, 13h
MOV AX, [BX][SI]



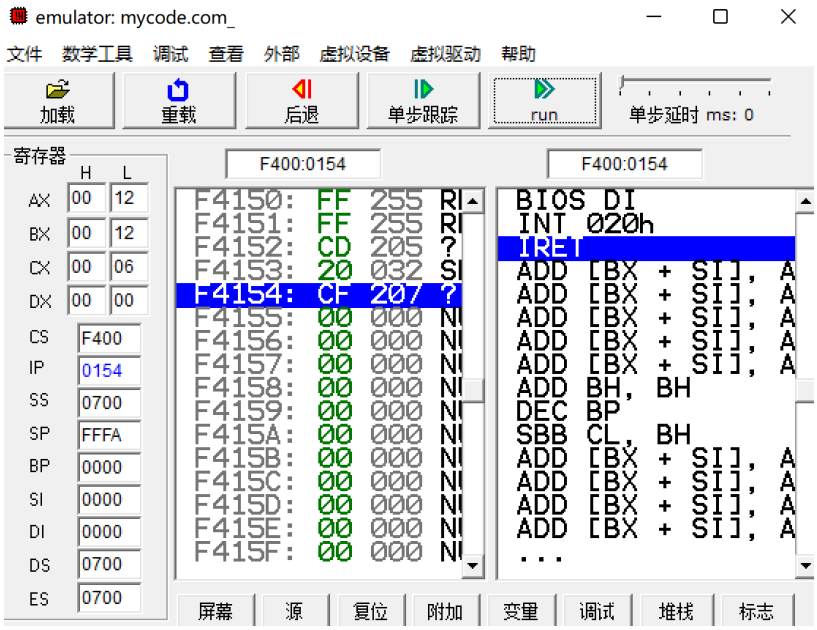
7）基址变址相对寻址：
MOV [138h], 38h
MOV SI, 25h
MOV BX, 13h
MOV AX, 100h[BX][SI]



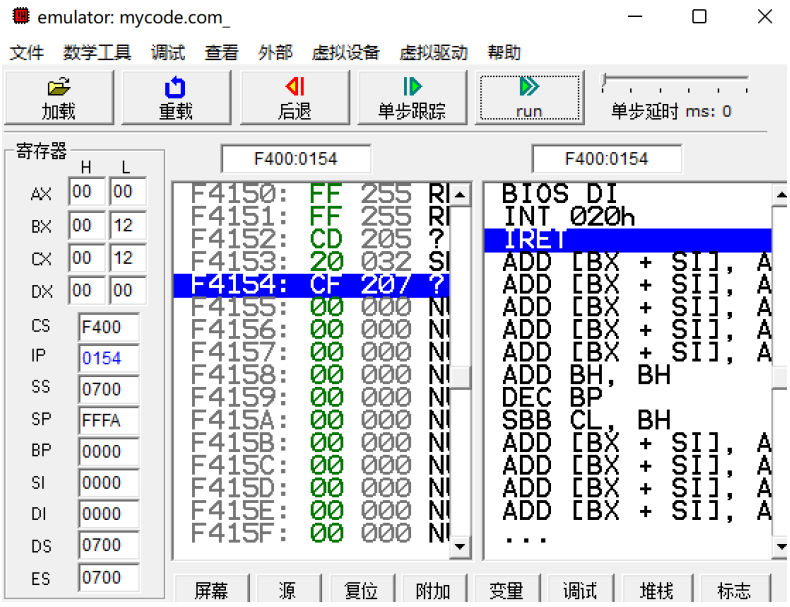
MOV [136h], 23h
MOV SI, 12h
MOV BX, 24h
MOV AX, 100h[BX][SI]



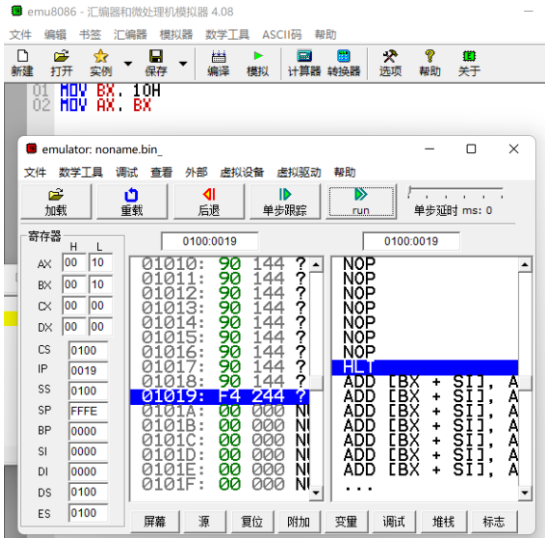
8）隐含寻址：
MOV AX, 12h
ADD BX, AX



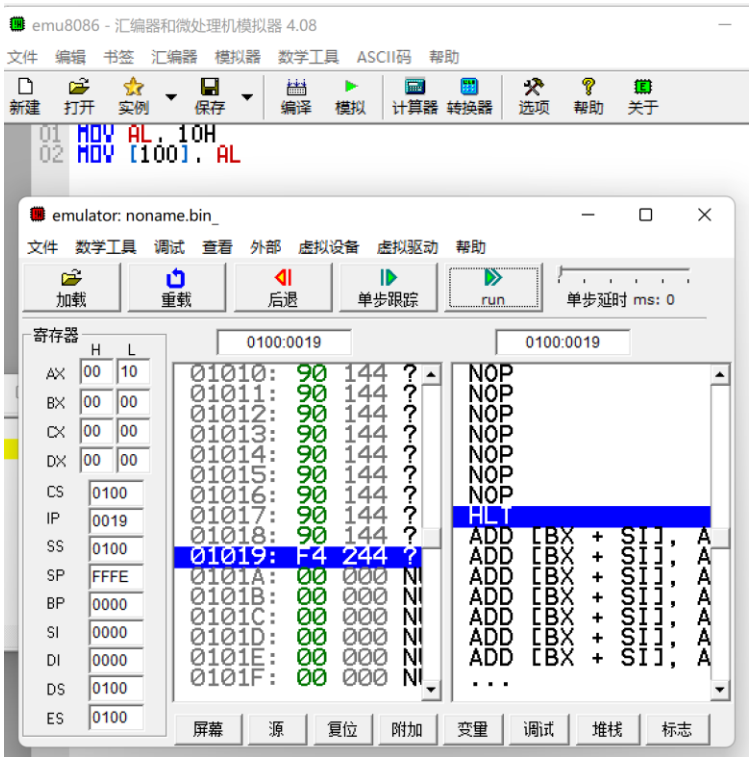
MOV CX, 12h
ADD BX, CX



3.指令系统
1)数据传送类指令
MOV AX, BX：将 BX 寄存器的内容移动到 AX 寄存器。

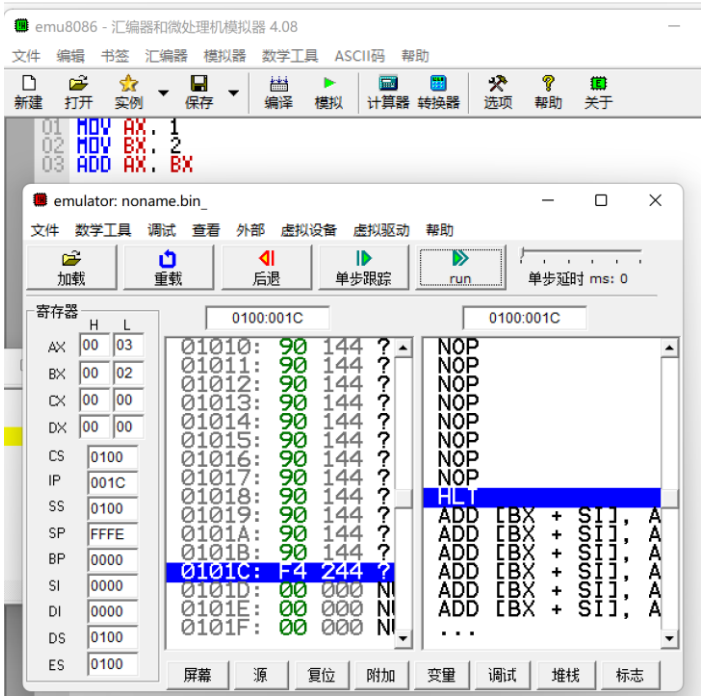


MOV [100], AL：将 AL 寄存器的内容移动到内存地址 100 处。

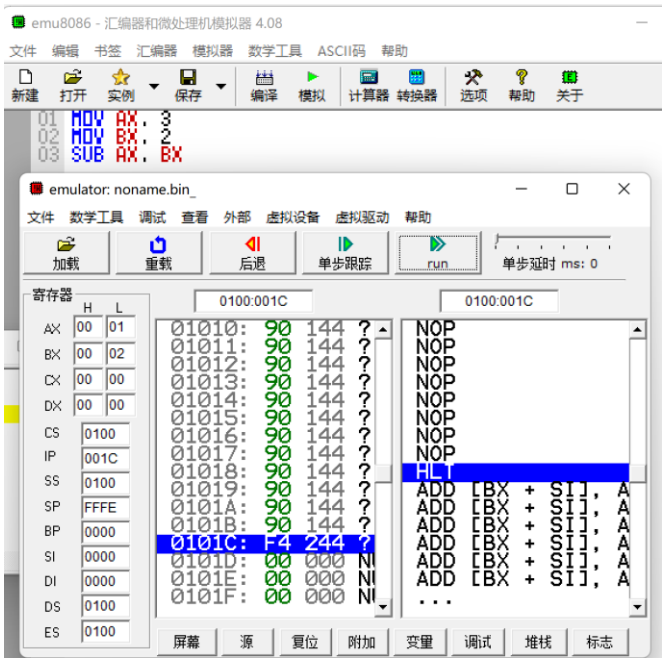


2)算术运算类指令

ADD AX, BX：将 BX 寄存器的内容加到 AX 寄存器上，结果存回 AX。

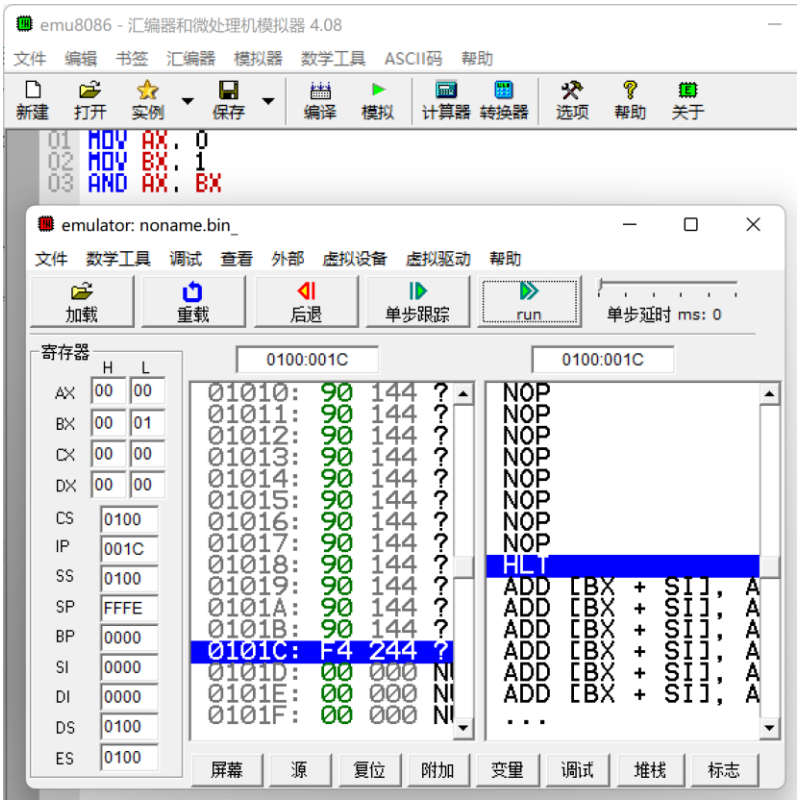


SUB AX, BX：从 AX 寄存器减去 BX 寄存器的内容，结果存回 AX，这会影响零标志 ZF、符号标志 SF 等。

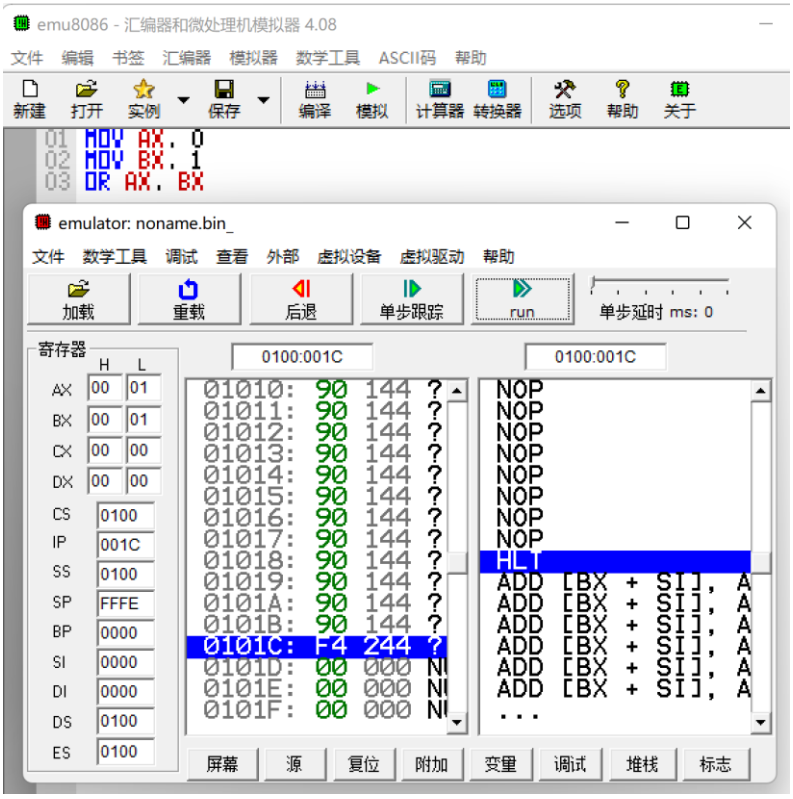


3)逻辑运算类指令

AND AX, BX：将 AX 和 BX 寄存器的内容进行逻辑与操作，结果存回 AX。



OR AX, BX：将 AX 和 BX 寄存器的内容进行逻辑或操作，结果存回 AX。



4)标志处理和 CPU 控制类指令

CLC：清除进位标志 CF。

STC：设置进位标志 CF。

5)移位和循环移位类指令

SHL AX, 1：将 AX 寄存器的内容左移一位。
ROL AX, 1：将 AX 寄存器的内容循环左移一位，最高位移至最低位。
6)处理器控制类指令
NOP：无操作，用于延迟执行或对齐代码。
HLT：处理器停止执行指令，直到接收到外部中断。
7)程序控制类指令
JMP label：无条件跳转到标签 label 处执行。
JZ label：如果零标志 ZF 为 1（即上一次操作结果为零），则跳转到标签 label 处执行。
8)输入/输出类指令
IN AX, 20：从端口 20 读取数据到 AX 寄存器。
OUT 20, AX：将 AX 寄存器的内容写入端口 20。

7. 编写一个简单的程序：将“This is my first ASM program-姓名（汉语拼音各人的姓名）”放在 DS=0700H，BX=0000H 开始的存储器单元中，然后将该内容搬移到 BX=0100H 开始的单元中，最后将该字符串通过 DOS 功能调用显示在屏幕上。

源代码如下:

```
ASSUME CS:CODE, DS:DATA, SS:STACK

data segment
    msg db "this is first ASM program-liaoxinyang$"
ends

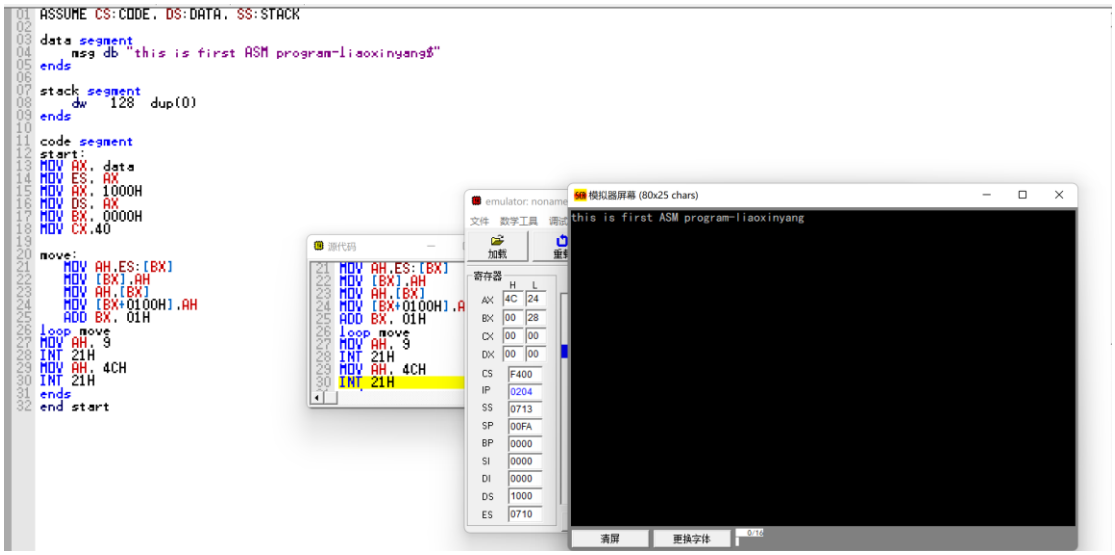
stack segment
    dw   128  dup(0)
ends

code segment
start:
MOV AX, data
MOV ES, AX
MOV AX, 1000H
MOV DS, AX
MOV BX, 0000H
MOV CX,40

move:
    MOV AH,ES:[BX]
    MOV [BX],AH
    MOV AH,[BX]
    MOV [BX+0100H],AH
    ADD BX, 01H
```

```
    loop move
    MOV AH, 9
    INT 21H
    MOV AH, 4CH
    INT 21H
    ends
end start
```

实验运行结果如下：



四、实验总结

通过本次实验，我深入学习了 EMU8086 汇编语言编程环境，并掌握了基本的汇编语言编程步骤。熟悉了 8086 的多种寻址方式，并理解了它们在实际编程中的应用。通过编写和观察不同类型指令的执行，我对寄存器、内存和处理器标志位的变化有了直观的认识。

此外，还学习了如何使用 DOS 功能调用进行人机交互，并掌握了汇编语言程序设计的基本原则。通过编写一个实际的程序，我将字符串在内存中的处理和显示过程具体化，加深了对汇编语言编程的理解。实验过程中，我也提高了解决问题的能力，并认识到了理论与实践相结合的重要性。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验二 数制转换与数值运算编程

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 04 月 20 日

成 绩

指导教师评语:
指导教师: ___年__月__日

一、 实验要求

1. 将 ASCII 码表示的十进制数转换为二进制数，自行绘制流程图并编写程序。十进制表示为：

$$D_n \times 10^n + D_{n-1} \times 10^{n-1} + \cdots + D_0 \times 10^0 = \sum_{i=0}^{n} D_i \times 10^i \qquad (1)$$

D_i 代表十进制数 0，1，2，…，9；
上式转换为：

$$\sum_{i=0}^{n} D_i \times 10^i = (\cdots((D_n \times 10 + D_{n-1}) \times 10 + D_{n-2}) \times 10 + \cdots + D_1) \times 10 + D_0 \qquad (2)$$

由式（2）可归纳十进制数转换为二进制数的方法：从十进制数的最高位 D_n 开始作乘 10 加次位的操作，依次类推，则可求出二进制数的结果。

表 1.1 数制对应关系表

十六进制	BCD 码	二进制机器码	ASCII 码	七段码	
				共阳	共阴
0	0000	0000	30H	40H	3FH
1	0001	0001	31H	79H	06H
2	0010	0010	32H	24H	5BH
3	0011	0011	33H	30H	4FH
4	0100	0100	34H	19H	66H
5	0101	0101	35H	12H	6DH
6	0110	0110	36H	02H	7DH
7	0111	0111	37H	78H	07H
8	1000	1000	38H	00H	7FH
9	1001	1001	39H	18H	67H
A		1010	41H	08H	77H
B		1011	42H	03H	7CH
C		1100	43H	46H	39H
D		1101	44H	21H	5EH
E		1110	45H	06H	79H
F		1111	46H	0EH	71H

2. BCD 码转换为二进制数

将四个二位十进制数的 BCD 码存放于 3500H 起始的内存单元中，将转换的二进制数存入 3510H 起始的内存单元中，自行绘制流程图并编写程序。

3. 两个非压缩 BCD 数加法程序。

要求：被加数和加数均为 4 位数，从键盘输入，分别放在 SBCD1 和 SBCD2 开始的地址单元中。运算结果放在 SSUM 开始的地址单元中。并在虚拟终端上以“SBCD1+SBCD2=SSUM”格式进行显示，自行绘制流程图并编写程序。

4. 从键盘上输入任意两个不大于 2 位数的正整数，计算其乘积。结果在屏幕上显示，自行绘制流程图并编写程序。

二、 实验目的

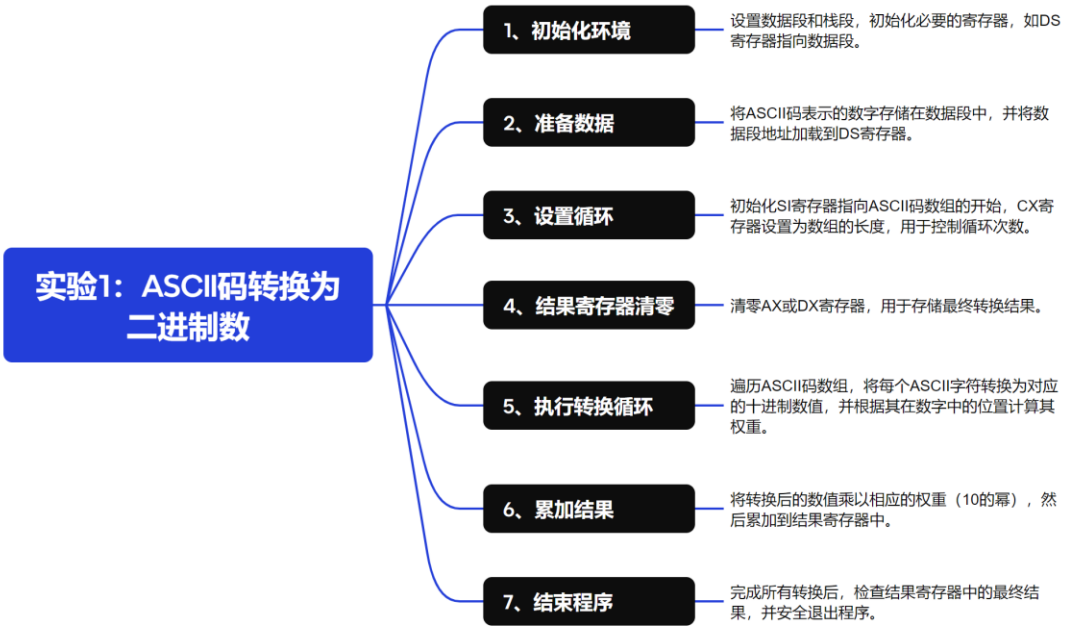
1. 掌握不同进制数及编码相互转换的程序设计方法，加深对数制转换的理解。

2. 熟悉在 EMU8086 集成环境中程序调试的方法。
3. 掌握使用运算类指令编程及调试方法。
4. 掌握运算类指令对各状态标志位的影响及其测试方法。
5. 学习使用软件监视变量的方法。

三、 实验代码及实验结果

实验一

1. 实验流程图：



2. 实验代码：

```
; 定义栈段
STACK SEGMENT STACK
    DW 64 DUP(?)
STACK ENDS

; 定义数据段
DATA SEGMENT
SADD DB 30H, 30H, 32H, 35H, 36H ; ASCII 码表示的数字 00256
DATA ENDS

; 定义代码段
CODE SEGMENT
ASSUME CS:CODE, DS:DATA

START:
    MOV AX, DATA ; 将数据段地址加载到 AX
```

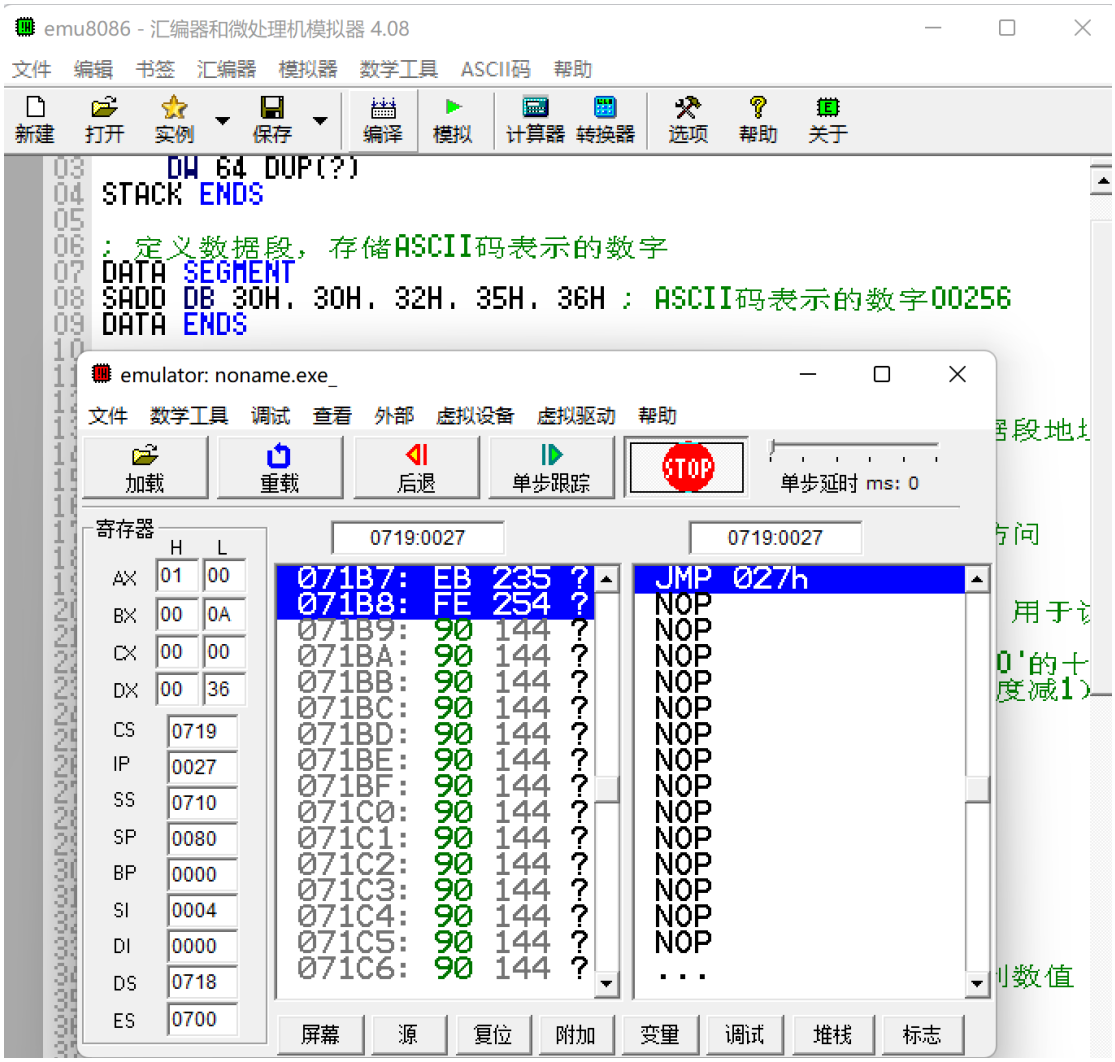
```
    MOV DS, AX    ; 将 AX 中的地址传递给 DS，使得数据段可访问

    MOV AX, OFFSET SADD
    MOV SI, AX
    MOV BX, 000AH ; BX 用于乘法操作，初始化为 10（ASCII '0'的十进制值）
    MOV CX, 0004H ; 设置 CX 为 4，用于控制循环次数（数组长度减 1）
    MOV AH, 00H   ; 清零 AH，用于清零高 8 位
    MOV AL, [SI]  ; 将数组的第一个字节加载到 AL

SUB AL, 30H
A1:
    IMUL BX
    MOV DX, [SI+01] ; 将 SI 指向的下一个字节加载到 DX
    AND DX, 00FFH   ; 清除 DX 的高 8 位，保留低 8 位
    ADC AX, DX
    SBB AX, 30H
    INC SI
    LOOP A1
A2: ; 无限循环，用于调试时的断点
    JMP A2

CODE ENDS
END START ; 程序结束，START 是程序的入口点
```

3. 实验结果：



实验二

1. 实验流程图：



2. 实验代码：

```
SSTACK SEGMENT STACK
   DW 64 DUP(?) ；定义栈段，预留 64 个 DWORD 空间
SSTACK ENDS

CODE SEGMENT
   ASSUME CS:CODE ；设置代码段寄存器

START:
   XOR AX, AX    ; 清零 AX
   MOV CX, 0004H ；设置循环计数为 4
   MOV SI, 3500H ；SI 指向 BCD 码数组起始地址

   ; 初始化 BCD 码数组
   MOV [SI], 01H
   MOV [SI+1], 02H
   MOV [SI+2], 03H
   MOV [SI+3], 04H

   ; 由于是四个二位十进制数，这里只初始化了前四个，需要补全
   MOV [SI+4], 05H
   MOV [SI+5], 06H
   MOV [SI+6], 07H
   MOV [SI+7], 08H

   MOV DI, 3510H ；DI 指向结果存储起始地址

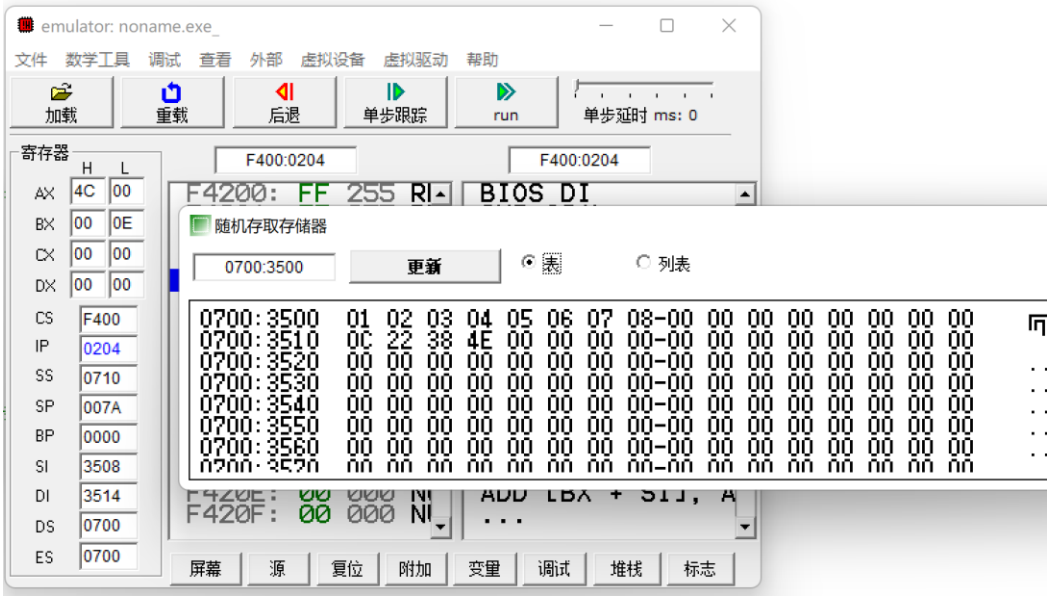
A1:
   MOV AL, [SI] ；从 SI 指向的内存地址加载 BCD 码到 AL
   ADD AL, AL
   MOV BL, AL
   ADD AL, AL
   ADD AL, AL
   ADD AL, BL
   INC SI
   ADD AL, [SI]
   MOV [DI], AL
   INC SI
   INC DI
   LOOP A1

   MOV AX, 4C00H ；设置程序退出代码
   INT 21H       ；调用中断，退出程序

CODE ENDS
```

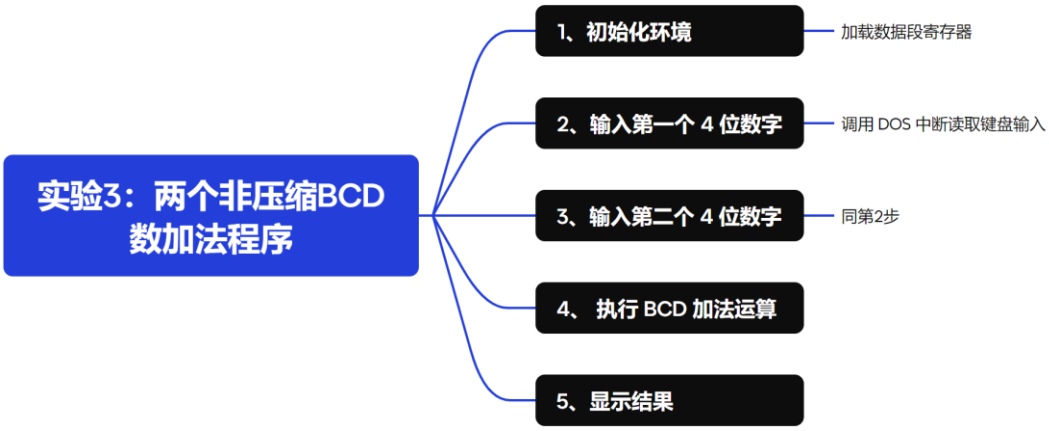
END START

3. 实验结果：



实验三

1. 实验流程图：



2. 实验代码：

```
DATA SEGMENT
   STRI1 DB 5 DUP(0) ；缓存第一个 4 位数，5 个字节
   STRI2 DB 5 DUP(0) ；缓存第二个 4 位数，5 个字节
   addc  DB '+'      ；加号字符
   equal DB '='      ；等号字符
DATA ENDS

CODE SEGMENT
   ASSUME CS:CODE, SS:STACK, DS:DATA, ES:DATA
```

```
START PROC FAR

PUSH DS
MOV AX, 0
PUSH AX
MOV AX, DATA
MOV DS, AX
MOV ES, AX

; 输入第一个 4 位数
MOV AH, 1H      ; 调用 DOS 中断读取键盘输入
INT 21H
SUB AL, 30H     ; 将 ASCII 码转换为数字
MOV STRI1+3, AL ; 存入 STRI1 的最低有效位
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI1+2, AL
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI1+1, AL
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI1, AL

; 换行
MOV DL, 0AH
MOV AH, 2H
INT 21H
MOV DL, 0DH
MOV AH, 2H
INT 21H

; 输入第二个 4 位数
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI2+3, AL
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI2+2, AL
```

```
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI2+1, AL
MOV AH, 1H
INT 21H
SUB AL, 30H
MOV STRI2, AL

; 换行
MOV DL, 0AH
MOV AH, 2H
INT 21H
MOV DL, 0DH
MOV AH, 2H
INT 21H

; 将 STRI1 中的数据压入堆栈
MOV SI, OFFSET STRI1
PUSH [SI]
PUSH [SI+1]
PUSH [SI+2]
PUSH [SI+3]

; 将 STRI2 中的数据压入堆栈
MOV DI, OFFSET STRI2
PUSH [DI]
PUSH [DI+1]
PUSH [DI+2]
PUSH [DI+3]

MOV CX, 5
CLC

; 循环执行加法运算
CYCLE:
   MOV AL, [SI]
   ADC AL, [DI]
   AAA      ; 调整结果为 BCD
   MOV [DI], AL
   INC SI
   INC DI
LOOP CYCLE
```

```
; 从堆栈中弹出并显示 STRI1 的值
MOV CX, 4
S:
   POP DX
   ADD DL, 30H
   MOV AH, 2
   INT 21H
LOOP S

; 显示加号
MOV DL, addc
MOV AH, 2
INT 21H

; 从堆栈中弹出并显示 STRI2 的值
MOV CX, 4
S2:
   POP DX
   ADD DL, 30H
   MOV AH, 2
   INT 21H
LOOP S2

; 显示等号
MOV DL, equal
MOV AH, 2
INT 21H

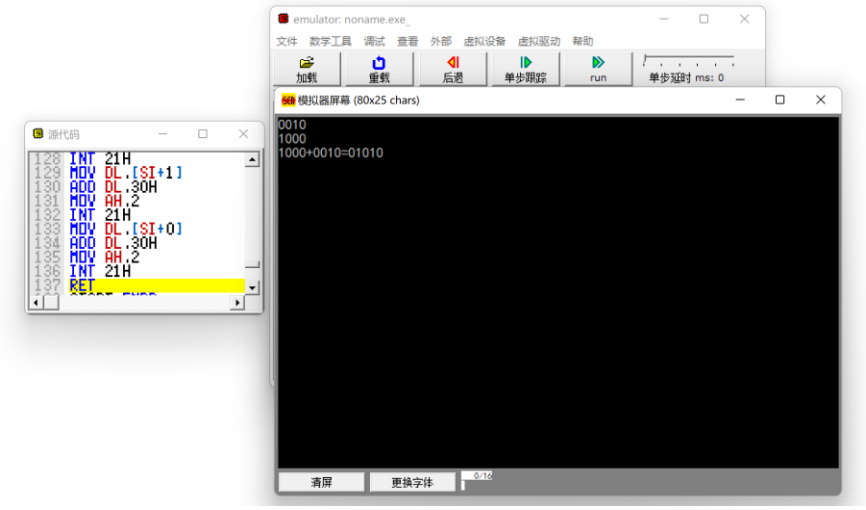
; 显示计算结果（SSUM）
MOV SI, OFFSET STRI2
MOV DL, [SI+4]
ADD DL, 30H
MOV AH, 2
INT 21H
MOV DL, [SI+3]
ADD DL, 30H
MOV AH, 2
INT 21H
MOV DL, [SI+2]
ADD DL, 30H
MOV AH, 2
INT 21H
MOV DL, [SI+1]
ADD DL, 30H
```

```
    MOV AH, 2
    INT 21H
    MOV DL, [SI+0]
    ADD DL, 30H
    MOV AH, 2
    INT 21H

    RET
START ENDP
CODE ENDS

    END START
```

3. 实验结果：



实验四

1. 实验流程图：



2. 实验代码：

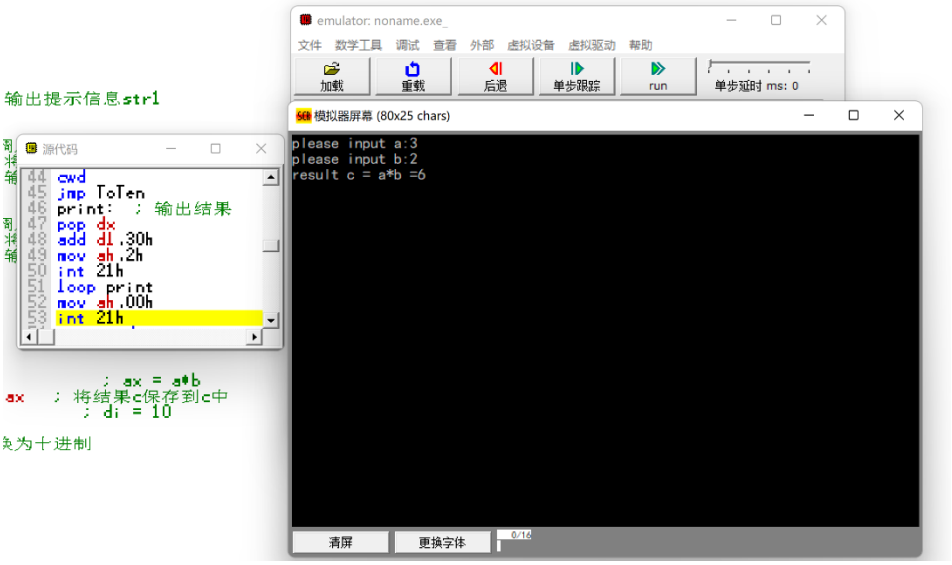
```
data segment
   str1 db 'please input a:$' ；提示输入 a 的字符串
   str2 db 'please input b:$' ；提示输入 b 的字符串
   str3 db 'result c = a*b =$'            ；输出结果的字符串
   a dw ？；定义变量 a，字类型
   b dw ？；定义变量 b，字类型
   c dw ？；定义变量 c，字类型
data ends

code segment
assume cs:code,ds:data
   main proc far
   start:
      mov ax,data
      mov ds,ax
      lea dx,str1 ；输出提示信息 str1
      mov ah,9h
      int 21h
      call input ；调用 input 过程输入 a
      mov a,bx    ；将输入的 a 保存到 a 中
      lea dx,str2 ；输出提示信息 str2
      mov ah,9h
      int 21h
      call input ；调用 input 过程输入 b
      mov b,bx    ；将输入的 b 保存到 b 中
      lea dx,str3 ；输出提示信息 str3
      mov ah,9h
      int 21h
      mov ax,a
      aad
      mov bx,ax
      mov ax,b
      aad
      mul bx                ; ax = a*b
      mov word ptr c,ax   ；将结果 c 保存到 c 中
      mov di,0ah           ; di = 10
      mov cx,0
   ToTen: ；将结果转换为十进制
      div di
      push dx
      inc cx
      cmp ax,0
      je print
      cwd
```

```
        jmp ToTen
    print: ; 输出结果
        pop dx
        add dl,30h
        mov ah,2h
        int 21h
    loop print
        mov ah,00h
        int 21h
    main endp
    input proc
        mov bx,0
        inputa:
        mov ah,1    ; 读取第一个字符
        int 21h
        sub al,30h
        mov bl,al
        mov ah,1       ; 读取第二个字符
        int 21h
        cmp al,0dh ; 比较是否按下 Enter 键
        je exit
        sub al,30h
        mov cl,8    ; 设定位移操作
        rol bx,cl
        mov bl,al
        jmp exit
    exit:
        call crlf
        ret
    input endp

    ; 输出换行
    crlf proc near
        mov dl,0ah
        mov ah,2h
        int 21h
        mov dl,0dh
        mov ah,2h
        int 21h
        ret
    crlf endp
code ends
    end start
```

3. 实验结果：



四、 实验总结

通过本次实验，我掌握了不同进制数及编码之间的转换方法，加深了对数制转换理论的理解。

在 EMU8086 集成环境中，我熟悉了程序调试的过程，学会了如何使用运算类指令进行编程和调试。

实验中，实现了 ASCII 码到二进制的转换、BCD 码到二进制的转换、两个非压缩 BCD 数的加法以及两个正整数乘积的计算，这些实践操作让我更加熟悉了汇编语言的编程技巧和程序设计的逻辑思维。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验三 顺序、分支、循环、子程序设计

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 4 月 27 日

成 绩

指导教师评语:

指导教师:

年 月 日

一、 实验要求

1. 编写 Q=a+(b-c)*d 的程序。（a、b、c、d 为 0-9 间的正整数）要求：a、b、c、d 均从键盘输入，计算结果在虚拟终端上显示。
2. 在 DS 数据段偏移地址为 DATA 开始的顺序 80 个单元中，存放着某班 40 个同学的微机原理考试成绩。请编程统计>=90 分，80 分-89 分，70 分-79 分，60 分-69 分以及<60 分的人数，将统计结果放在当前数据段偏移地址为 BUFFER 的顺序单元中。并在虚拟终端上显示各分数段的人数。
3. 从键盘上输入 1 个数（不大于 5 位，以回车键结束），判断其奇偶性，如果是奇数，屏幕上显示”It is odd”，否则显示”It is even”。
4. 从键盘上输入 N 个字符（N<16），求这 N 个字符中’A’的个数，并将’A’的个数显示在屏幕上。
5. 已知 BUF1 中有 N1 个按从小到大的顺序排列的互不相等的无符号数，BUF2 中有 N2 个从小到大的顺序排列的互不相等的无符号数。编写程序将 BUF1 和 BUF2 中的数合并到 BUF3 中，使在 BUF3 中存放的数互不相等且按从小到大的顺序排列。

二、 实验目的

学习顺序、分支、循环结构以及子程序的编程设计。

三、 实验代码及实验结果

1.

实验代码如下：

```
data segment
        prompt1 db 'number a:$'
        prompt2 db 'number b:$'
        prompt3 db 'number c:$'
        prompt4 db 'number d:$'
        result_msg db 'a+(b-c)*d=$'
        a dw 0
        b dw 0
        c dw 0
        d dw 0
        result dw 0    ; 结果存储在  result  中
data ends

code segment
        main proc
                assume cs:code,ds:data
start:
         mov ax,data
         mov ds,ax
         lea dx,prompt1
```

```
        mov ah,9h
        int 21h
        call input ; 输入 a
        mov a,bx
        lea dx,prompt2
        mov ah,9h
        int 21h
        call input ; 输入 b
        mov b,bx
        lea dx,prompt3
        mov ah,9h
        int 21h
        call input ; 输入 c
        mov c,bx
        lea dx,prompt4
        mov ah,9h
        int 21h
        call input ; 输入 d
        mov d,bx
        lea dx,result_msg
        mov ah,9h
        int 21h      ; 输出结果提示
        ; 计算过程
        mov ax,b
        sub ax,c
        mov bx,ax
        mov ax,d
      ; 计算 (b-c)*d
        imul bx     ; 16 位乘法，结果放在 AX 寄存器中
        add ax,a    ; 加上 a
        mov result,ax
        ; 输出结果
        test ax,8000h
        jz   print_positive     ; 如果是正数
        jnz print_negative      ; 如果是负数

print_negative:
                mov   dl,'-'
                mov ah,02h      ; 显示输出。入口参数：AH=02H; DL=要显
示的字符
                int 21h
                mov ax,result
                neg ax
                call print_number
```

```
                jmp exit_program

print_positive:
                mov ax,result
                call print_number
                jmp exit_program

print_number:
                mov di,10 ; DI = 10
                mov cx,0
convert_to_decimal:
                div di
                push dx
                inc cx
                cmp ax,0
                jne convert_to_decimal

output_number:
                pop dx
                add dl,30h
                mov ah,02h
                int 21h
                loop output_number
                ret

exit_program:
            mov ax,4c00h    ; 程序中止
            int 21h

main endp

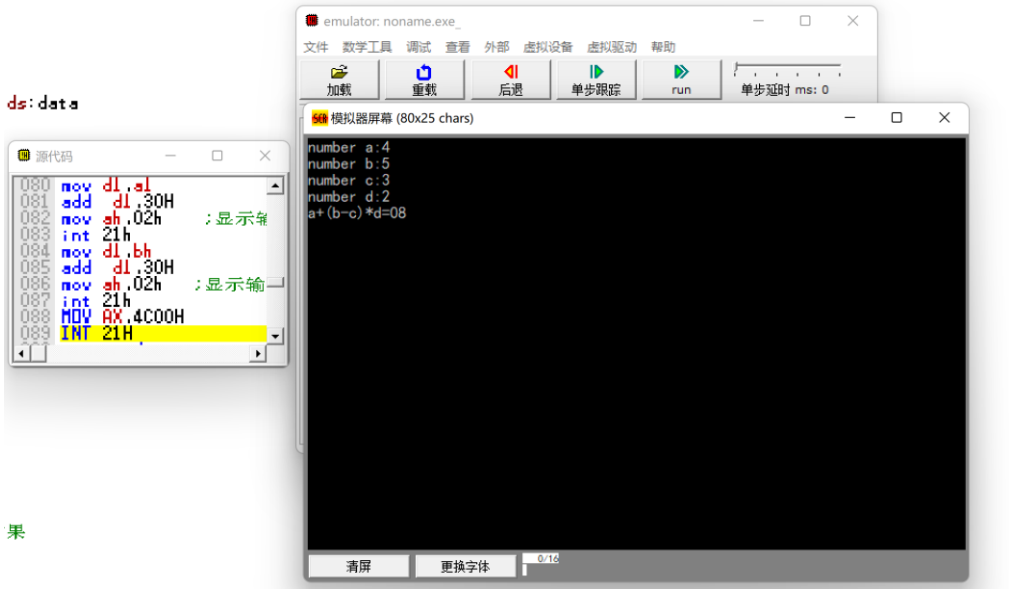
; 输入子程序
input proc
      mov bx,0    ; 清空 BX
      input_loop:
            mov ah,01h ; 输入一个字符，AL 等于输入的字符
            int 21h
            cmp al,0dh ; 检查是否为回车
            je input_done
            sub al,30h ; 转换为十进制
            mov cl,4      ; 左移 4 位
            shl bx,cl    ; BX 左移 4 位
            add bl,al    ; 将 AL 加到 BX 低位
            jmp input_loop
```

```
        input_done:
            call newline
            ret
input endp

; 换行操作
newline proc
        mov dl,0ah ; 换行
        mov ah,2h
        int 21h
        mov dl,0dh ; 回车
        mov ah,2h
        int 21h
        ret
newline endp

code ends
    end start
```

实验结果如下所示：



2.

实验代码如下：

```
data segment
        scores db 58, 76, 82, 91, 63, 57, 88, 74, 87, 69, 92, 71, 64, 78, 85, 93, 60,
59, 77, 80, 81, 72, 65, 89, 90, 73, 66, 75, 86, 56, 68, 84, 95, 79, 83, 61, 55, 49, 50
        buffer db 5 dup(0)   ; 用于存储各分数段的计数
        str1    db " >=90 : $"
        str2    db " 80-89 : $"
```

```
        str3    db " 70-79 : $"
        str4    db " 60-69 : $"
        str5    db " <60   : $"
data ends

code segment
        assume CS:code, DS:data

main:
        mov ax, data
        mov ds, ax

        ; 初始化计数器
        xor dx, dx
        mov bx, dx
        mov cx, 40                  ; 成绩数量

        ; 遍历成绩数组，统计各分数段
        mov si, offset scores
count_scores:
        mov al, [si]
        cmp al, 90
        jc eighty
        inc dh                      ; >=90
        jmp stor
        eighty:
        cmp al, 80
        jc seventy
        inc dl                      ; 80-89
        jmp stor
        seventy:
        cmp al, 70
        jc sixty
        inc bh                      ; 70-79
        jmp stor
        sixty:
        cmp al, 60
        jc below_sixty
        inc bl                      ; 60-69
        jmp stor
        below_sixty:
        inc bp                      ; <60
        stor:
        inc si
```

```
        loop count_scores

        ; 存储统计结果到 buffer
        mov [buffer], bp
        mov [buffer+1], bl
        mov [buffer+2], bh
        mov [buffer+3], dl
        mov [buffer+4], dh

        ; 显示成绩段统计
        call displayScoresSummary

        ; 退出程序
        mov ax, 4c00h
        int 21h

displayScoresSummary proc near
        ; 显示成绩段统计信息
        lea dx, str1
        mov ah, 09h
        int 21h
        mov al, [buffer+4] ; >=90
        call printf
        call crlf

        lea dx, str2
        mov ah, 09h
        int 21h
        mov al, [buffer+3] ; 80-89
        call printf
        call crlf

        lea dx, str3
        mov ah, 09h
        int 21h
        mov al, [buffer+2] ; 70-79
        call printf
        call crlf

        lea dx, str4
        mov ah, 09h
        int 21h
        mov al, [buffer+1] ; 60-69
        call printf
```

```
        call crlf

        lea dx, str5
        mov ah, 09h
        int 21h
        mov al, [buffer]      ; <60
        call printf
        call crlf
        ret
displayScoresSummary endp

; 以下是 printf, crlf 过程的实现，用于数字显示和换行。
; ...

printf proc near ; 显示 AL 寄存器中的数字，并跟随换行
        push cx
        push bx
        push dx
        mov cx, 10          ; 10 进制
        mov bx, 10          ; 除数
        mov dx, 0           ; 余数

print_next_digit:
        xor dx, dx          ; 清零 DX，准备存储余数
        div bx              ; AX / BX，商在 AX，余数在 DX
        add dl, '0'         ; 将余数转换为 ASCII 码
        push ax
        mov ah, 02h         ; DOS 服务号 - 显示字符
        int 21h
        pop ax
        test ax, ax         ; 检查 AX 是否为 0
        jnz print_next_digit

        pop dx
        pop bx
        pop cx
        ret
printf endp

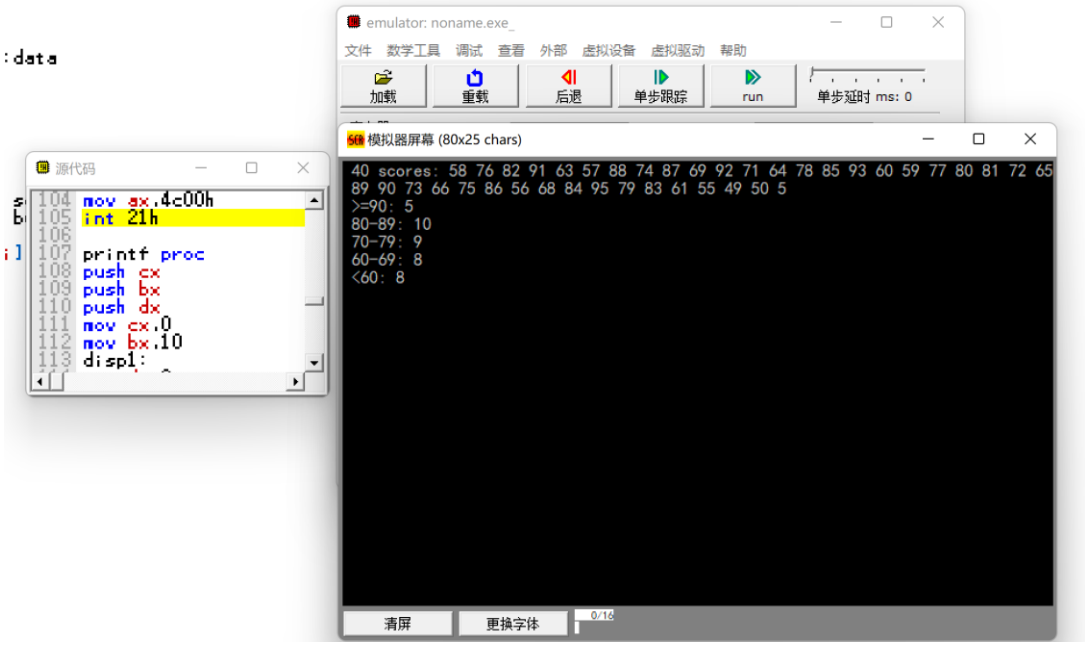
crlf proc near      ; 换行
        push ax
        mov dl, 0Dh         ; 回车符
        mov ah, 02h         ; DOS 服务号 - 显示字符
        int 21h
```

```
        mov dl, 0Ah       ; 换行符
        int 21h
        pop ax
        ret
crlf endp

space proc          ; 打印空格
        push ax
        mov dl, ' '
        mov ah, 02h
        int 21h
        pop ax
        ret
space endp

code ends
end main
```

实验结果如下所示：



3.

实验代码如下：

```
data segment
        odd db 'It is odd$'
        even db 'It is even$'
data ends

stack segment
        dw 64 dup(0)
ends
```

```
code segment
start:
        mov ax,data
        mov ds,ax
        mov bx,0
        input1:
                mov ah,01h
                int 21h         ;键盘输入一个数
                cmp al,0dh
                jz select       ;数据写入后进行奇数偶数判断
                sub al,30h
                mov cl,al
                mov ch,00
                mov ax,bx
                mov bx,0ah
                mul bx
                mov bx,ax
                add bx,cx
                jmp input1
        select:
                mov bx,0
                mov ah,2
                mov dl,0ah      ;回车后进行判断
                int 21h
                mov dl,0dh
                int 21h

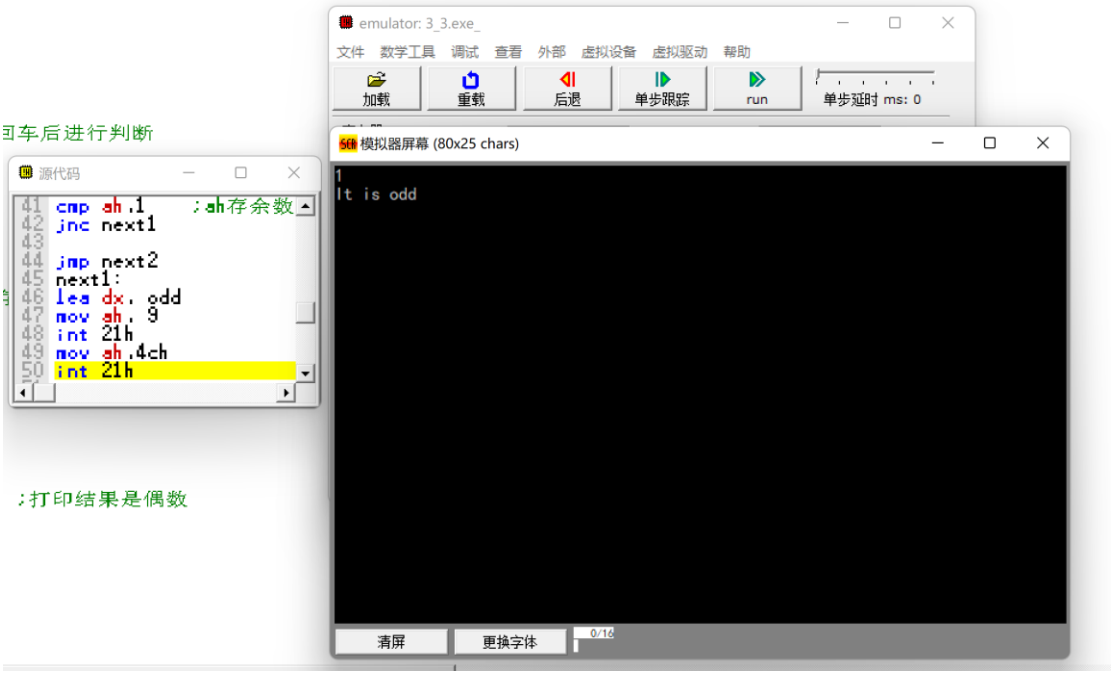
                sub ah,ah
                mov al,cl
                mov bl,2
                div bl
                cmp ah,1        ;ah 存余数
                jnc next1

                jmp next2
        next1:                  ;打印结果是奇数
                  lea dx, odd
                  mov ah, 9
                  int 21h
                  mov ah,4ch
                  int 21h

        next2:                  ;打印结果是偶数
```

```
lea dx, even
mov ah, 9
int 21h
mov ah,4ch
int 21h
```

实验结果如下所示：



4.

实验代码如下：

```
stack segment
        dw 120 dup(0)
ends

code segment
        start:
        mov ds, ax
        mov bx, 0
        mov dl, 0

        input:
        mov ah, 01h
        int 21h
        cmp al, 0dh             ;
        jz write
        cmp al, 'A'
        je next
        jmp input
```

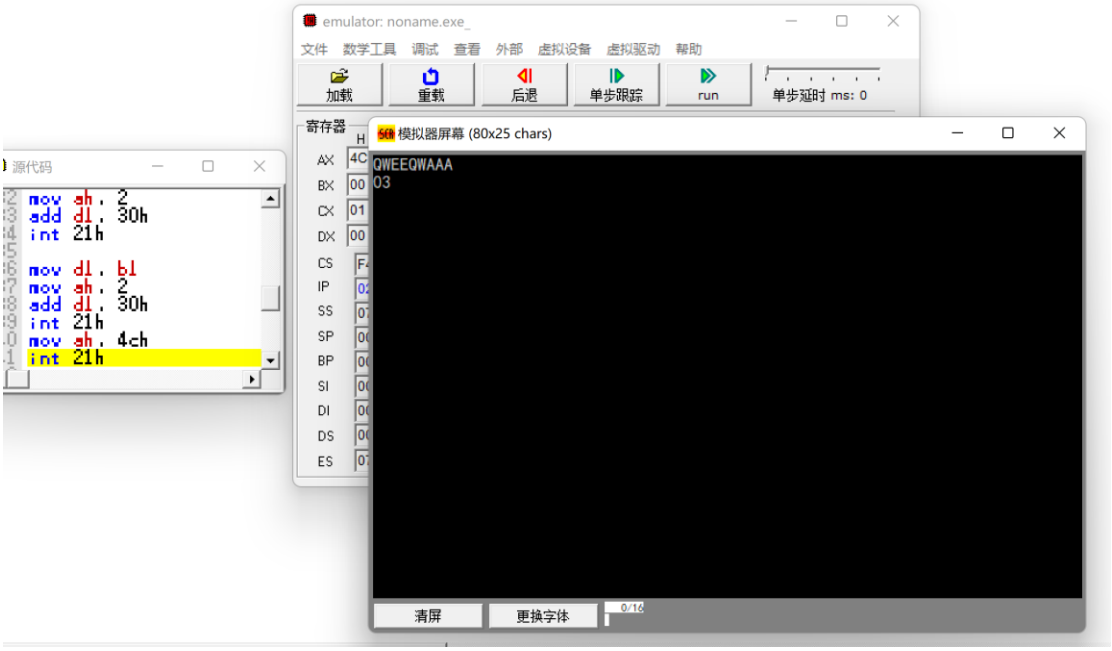
```
write:
mov ah, 2
mov dl, 0ah
int 21h
mov dl, 0dh
int 21h
mov ax, bx
mov dl, 10
div dl

mov dl, al
mov bl, ah
mov ah, 2
add dl, 30h
int 21h

mov dl, bl
mov ah, 2
add dl, 30h
int 21h
mov ah, 4ch
int 21h

next:
add bx, 1
jmp input
```

实验结果如下所示：



5.
实验代码如下：

```
data segment
    BUF1 db 1,3,5,7,9,
    BUF2 db   2,4,6,8,
    BUF3 db 32 dup(204)
    N1      db 6
    N2      db 5
    string db 'Array merged completed!$'
data ends

code segment
    ASSUME CS:CODE,DS:DATE
    start:
        mov ax,data
        mov ds,ax
        lea si,BUF1 ;pointer of buffer1
        lea di,BUF2 ;pointer of buffer2
        lea bx,BUF3 ;pointer of buffer3
        mov ax,0
        mov cx,0
        mov dx,0
        mov ah,N1 ;number of N1
        mov al,N2 ;number of N2
        mov cl,N1
        add cl,N2 ;number of N1+N2
        mov dx,word ptr N2 ;number of N1+N2-1
        dec dx
        mov dh,0
        add dl,ah

        mov ax,word ptr N1 ;number of N1-1
        dec ax
        mov ah,0
    while:
        cmp si,ax
        ja b2     ;′óóúì?×a
        cmp di,dx
        ja b1
        push dx
        mov dx,[di]
        cmp [si],dl
        pop dx
        ja b2             ;buf1[si] > buf2[di]
```

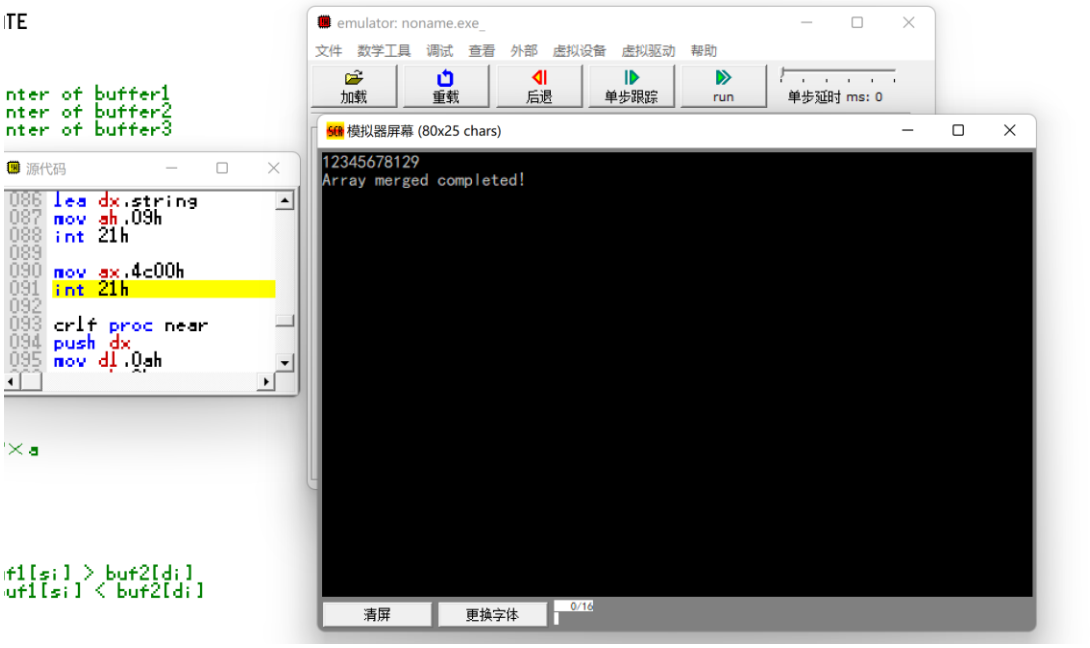
```
        jb b1               ;buf1[si] < buf2[di]
        push dx
        push ax
        mov dx,[si]
        mov [bx],dl         ;buf1[si] == buf2[di]
        add dl,30h
        mov ah,2h
        int 21h
        pop ax
        pop dx
        inc si
        inc di
        inc bx
        dec cx
        jmp tail
b2:
        push dx
        push ax
        mov dx,[di]
        mov [bx],dl
        add dl,30h
        mov ah,2h
        int 21h
        pop ax
        pop dx
        inc bx
        inc di
        jmp tail
b1:
        push dx
        push ax
        mov dx,[si]
        mov [bx],dl
        add dl,30h
        mov ah,2h
        int 21h
        pop ax
        pop dx
        inc bx
        inc si
tail:
loop while
        call crlf
        lea dx,string
```

```
        mov ah,09h
        int 21h

        mov ax,4c00h
        int 21h

    crlf proc near
        push dx
        mov dl,0ah
        mov ah,2h
        int 21h
        mov dl,0dh
        mov ah,2h
        int 21h
        pop dx
        ret
    crlf endp
code ends
end start
```

实验结果如下所示：



四、 实验总结

通过本次汇编语言编程实验，我深入理解了顺序、分支和循环结构的重要性，并掌握了子程序设计的基本方法。

实验中，我首先实现了一个简单的数学表达式计算程序，加深了对输入输出处理的理解。接着，我完成了成绩统计任务，学习了如何使用分支和循环来处理实际问题。在奇偶性判断和字符计数的任务中，我进一步熟悉了条件判断和循环的应用。最后，通过合并排序数组的练习，我掌握了数组操作和算法实现

的技巧。整个实验不仅提升了我的编程能力，也加深了我对汇编语言特性的认识，为后续更复杂的程序设计打下了坚实的基础。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验四 综合性汇编程序设计

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 5 月 18 日

成 绩

指导教师评语:
指导教师: 年 月 日

一、 实验要求

1. 计算 S＝1＋2×3＋3×4＋4×5＋…＋N（N＋1），直到 N（N＋1）项大于 200 为止。
2. 求 N!。N 为键盘输入的不大于 8 的正整数。
3. 从键盘输入一行字符（以回车结束），并按字母、数字及其它字符分类计数，最后显示出这 3 个计数结果。
4. 编写一电子钟程序，在屏幕正中按以下格式显示：

YYYY 年 MM 月 DD 日

HH：MM：SS

二、 实验目的

综合多种汇编程序设计方法，掌握具有人机交互界面的较复杂的汇编应用程序的编写。

一、 实验代码及实验结果

1.

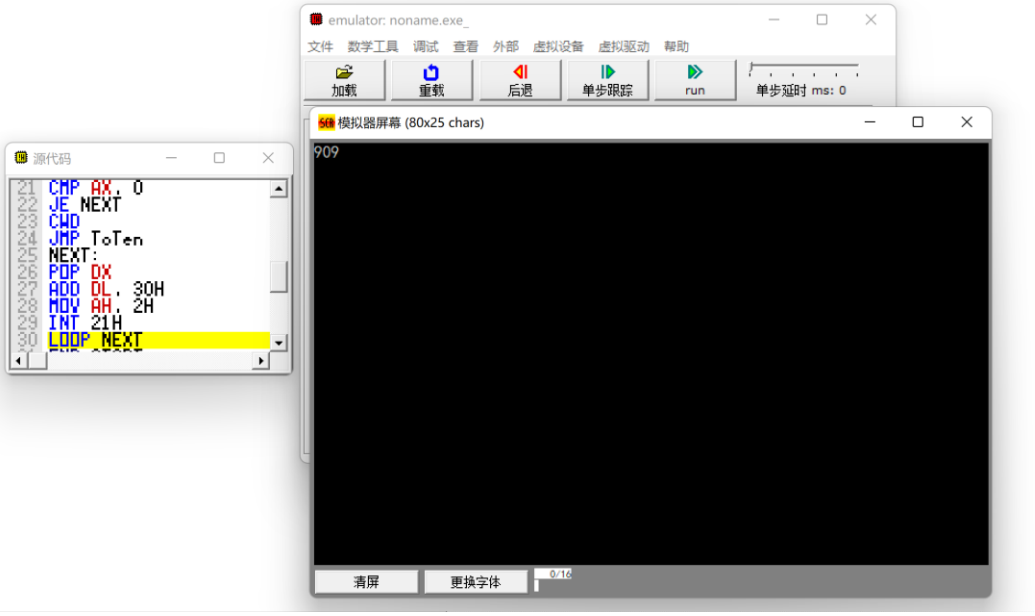
实验代码如下：

```
START:
    MOV DX, 0001H
    MOV BX, 0002H
NEXT1:
    MOV AL, BL
    INC BL
    MUL BL
    ADD DX, AX
    CMP AX, 00C8H
    JNA NEXT1

    SUB DX, AX
    MOV AX, DX
    MOV DX, 0000H
    MOV DI, 0AH
    MOV CX, 0
    ToTen:
    DIV DI
    PUSH DX
    INC CX
    CMP AX, 0
    JE NEXT
    CWD
    JMP ToTen
NEXT:
    POP DX
```

```
        ADD DL, 30H
        MOV AH, 2H
        INT 21H
        LOOP NEXT
END START
```

实验结果如下：



2.

实验代码如下：

```
DATA SEGMENT
        STR1 DB "N should <=8&&>0!$"
        DATA1 DB 0
DATA ENDS

START:
        MOV CX, 0
        MOV AX, DATA
        MOV DS, AX
        MOV SI, OFFSET DATA1

INPUT:
        MOV AH, 01H
        INT 21H
        INC CX
        CMP AL, 38H
        JA ERROR
        MOV [SI], AL
        INC SI
        CMP AL, 0DH
```

```
        JNE INPUT
        CMP CX, 02H
        JE NEXT

ERROR:
        MOV DL, 0AH
        MOV AH, 02H
        INT 21H
        MOV DL, 0DH
        MOV AH, 02H
        INT 21H

        MOV DX, OFFSET STR1
        MOV AH, 09H
        INT 21H

        MOV AH, 00H
        INT 21H

NEXT:
        MOV AL, [SI-2]
        CMP AL, 30H
        JE ERROR
        SUB AL, 30H
        MOV AH, 00H
        MOV BX, AX

        MOV DL, 0AH
        MOV AH, 02H
        INT 21H

        MOV DL, 0DH
        MOV AH, 02H
        INT 21H

        MOV AX, BX

NEXT1:
        CMP BX, 0001H
        JE NEXT2
        DEC BX
        MUL BX
        CMP BX, 0001H
        JA NEXT1
```

```
    JMP NEXT2

NEXT2:
    MOV CX, 0
    MOV DX, 0000H
    MOV DI, 0AH

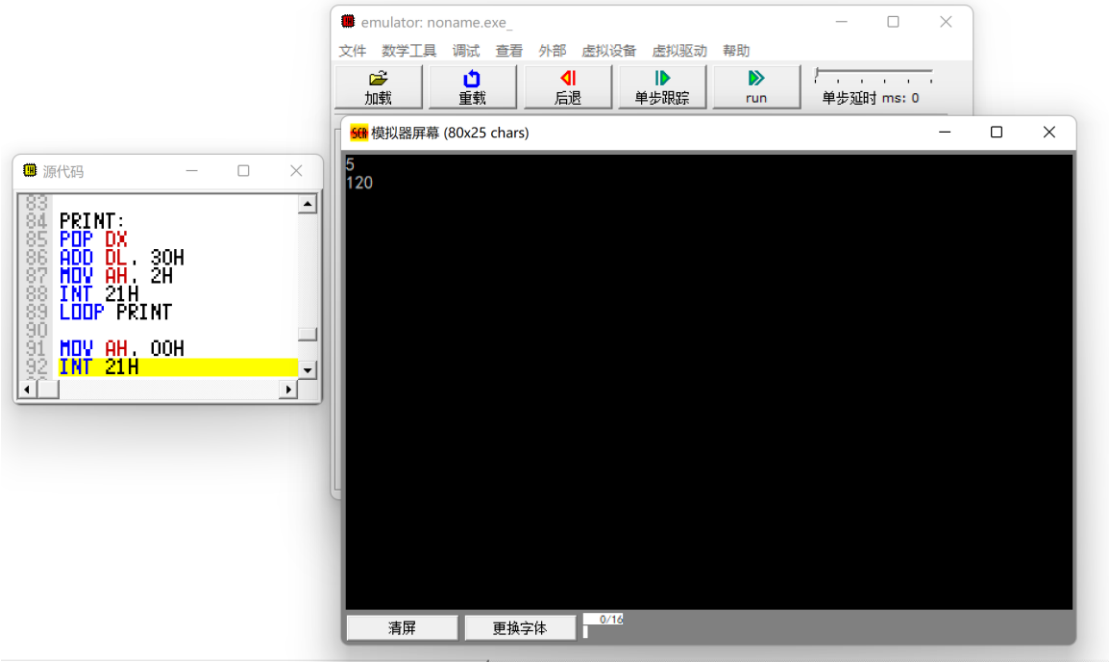
ToTen:
    DIV DI
    PUSH DX
    INC CX
    CMP AX, 0
    JE PRINT
    CWD
    JMP ToTen

PRINT:
    POP DX
    ADD DL, 30H
    MOV AH, 2H
    INT 21H
    LOOP PRINT

    MOV AH, 00H
    INT 21H

END START
```

实验结果如下：



3.

实验代码如下：

```
DATA SEGMENT
        STR1 DB "NUM OF ALPHABET :$"
        STR2 DB "NUM OF NUMBER :$"
        STR3 DB "NUM OF OTHERS :$"
        ALPH DB 0
        NUM DB 0
        OTHERS DB 0
    ENDS

    CODE SEGMENT
        ASSUME CS:CODE, DS:DATA

    START:
        MOV AX, DATA
        MOV DS, AX
        MOV BX, 0
        MOV SI, OFFSET ALPH

    INPUT:
        MOV AH, 01H
        INT 21H
        CMP AL, 0DH
        JE PRINT
        CMP AL, 30H
```

```
        JB ADDOTHERS
        CMP AL, 3AH
        JB ADDNUM
        CMP AL, 41H
        JB ADDOTHERS
        CMP AL, 5BH
        JB ADDALPH
        CMP AL, 61H
        JB ADDOTHERS
        CMP AL, 7BH
        JB ADDALPH
        JMP ADDOTHERS

PRINT:
        CALL CALF
        LEA DX, STR1
        MOV AH, 09H
        INT 21H
        CALL PRINT1
        INC SI
        CALL CALF

        LEA DX, STR2
        MOV AH, 09H
        INT 21H
        CALL PRINT1
        INC SI
        CALL CALF

        LEA DX, STR3
        MOV AH, 09H
        INT 21H
        CALL PRINT1
        MOV AX, 00H
        INT 21H

PRINT1:
        MOV CX, 0
        MOV DX, 0
        MOV DI, 0AH
        MOV AL, [SI]
        MOV AH, 00H
ToTen:
        DIV DI
```

```
        PUSH DX
        INC CX
        CMP AX, 0
        JE NEXT1
        CWD
        JMP ToTen

NEXT1:
        POP DX
        ADD DL, 30H
        MOV AH, 02H
        INT 21H
        LOOP NEXT1

RET

ADDALPH:
        MOV DL, [SI]
        INC DL
        MOV [SI], DL
        JMP INPUT

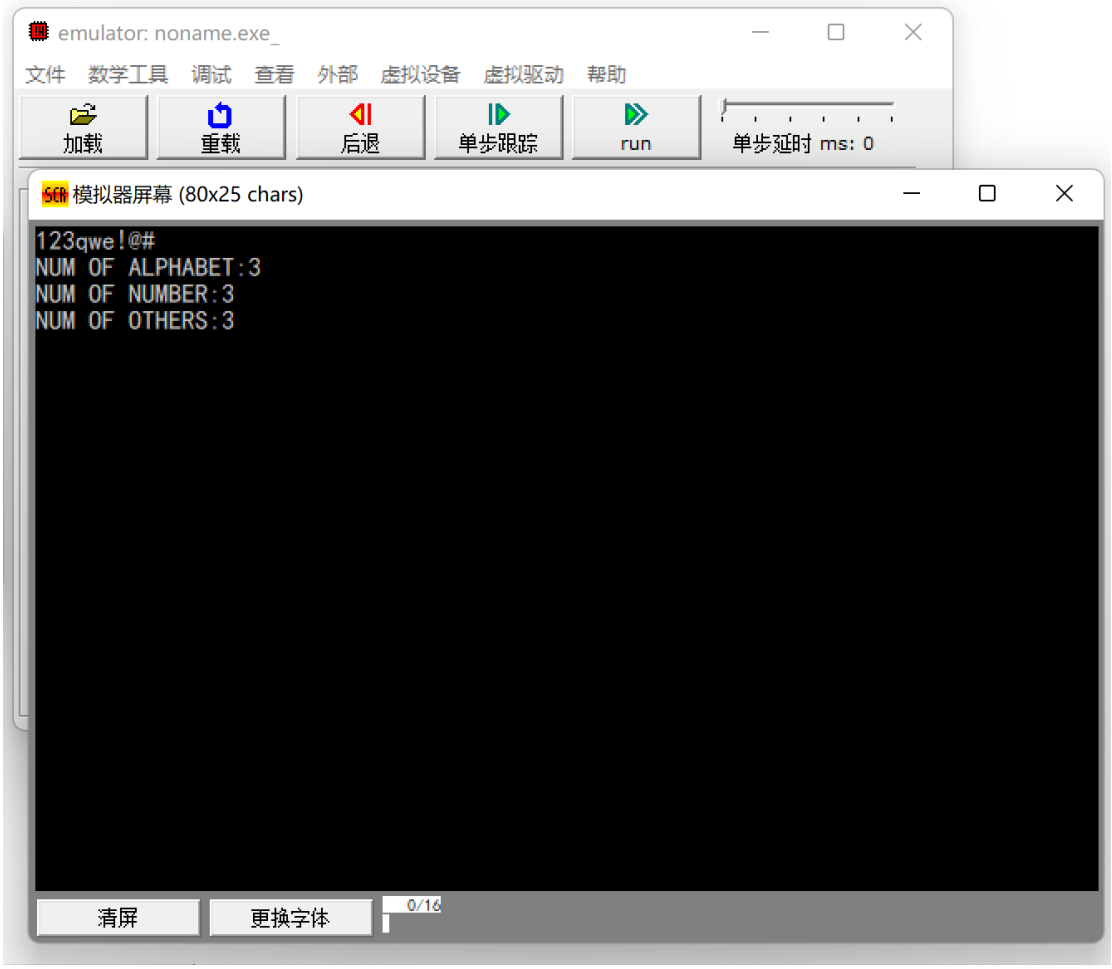
ADDNUM:
        MOV DL, [SI+1]
        INC DL
        MOV [SI+1], DL
        JMP INPUT

ADDOTHERS:
        MOV DL, [SI+2]
        INC DL
        MOV [SI+2], DL
        JMP INPUT

CALF:
        MOV DL, 0AH
        MOV AH, 02H
        INT 21H
        MOV DL, 0DH
        MOV AH, 02H
        INT 21H
RET

ENDS
```

END START
实验结果如下：



4.
实验代码如下：

```
DATA SEGMENT
    ; 定义时钟格式信息变量
    YEAR DW 0
    MONTH DB 0
    DAY DB 0
    HOUR DB 0
    MINUTE DB 0
    SECOND DB 0
    TIME1 DB "2024year 5month 18day$"
    TIME2 DB "20:10:10$"
    LEN1 EQU $-TIME1
    LEN2 EQU $-TIME2
    NUM DB 0
ENDS
```

```
CODE SEGMENT
START:
    MOV AX, DATA
    MOV DS, AX
    MOV ES, AX

BEGIN:
    CALL SETSHOW1
    CALL GET_SYSTEM_TIME1
    CALL SETSHOW2
    CALL GET_SYSTEM_TIME2
    JMP BEGIN

GET_SYSTEM_TIME1 PROC
    ; 获取系统日期
    MOV AH, 2AH
    INT 21H
    MOV YEAR, CX
    MOV MONTH, DH
    MOV DAY, DL
    ; 将年、月、日转换为字符串并存储在 TIME1
    ; ...
    ; 输出系统日期
    LEA DX, TIME1
    MOV AH, 9
    INT 21H
    RET
GET_SYSTEM_TIME1 ENDP

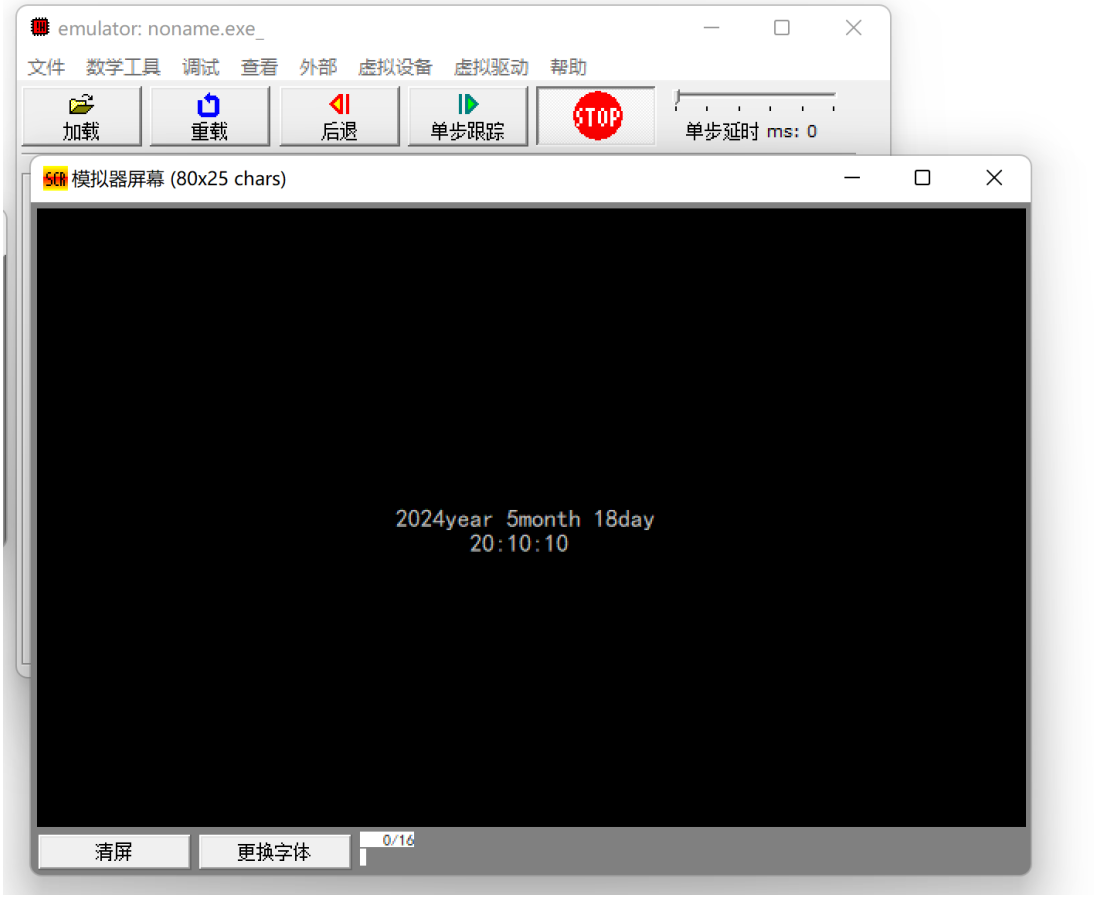
GET_SYSTEM_TIME2 PROC
    ; 获取系统时间
    MOV AH, 2CH
    INT 21H
    MOV HOUR, CH
    MOV MINUTE, CL
    MOV SECOND, DH
    ; 将时、分、秒转换为字符串并存储在 TIME2
    ; ...
    ; 输出系统时间
    LEA DX, TIME2
    MOV AH, 9
    INT 21H
    RET
GET_SYSTEM_TIME2 ENDP
```

```
SETSHOW1 PROC
    ; 设置光标位置以显示日期
    MOV DH, 12
    MOV DL, 29
    MOV BH, 0
    MOV AH, 2
    INT 10H
    RET
SETSHOW1 ENDP

SETSHOW2 PROC
    ; 设置光标位置以显示时间
    MOV DH, 13
    MOV DL, 35
    MOV BH, 0
    MOV AH, 2
    INT 10H
    RET
SETSHOW2 ENDP

    MOV AX, 4C00H
    INT 21H
ENDS
END START
```

实验结果如下：



四、 实验总结

通过本次汇编语言编程实验，我深入实践了多种程序设计方法，包括顺序、分支、循环结构以及子程序的应用。

实验中，我首先完成了一个累计求和程序，该程序计算序列 S 的值直到其超过 200，这锻炼了我的循环控制能力。紧接着，我编写了一个计算阶乘的程序，加深了对递归和循环的理解。此外，我还实现了一个字符分类计数程序，它提高了我对条件判断和数组操作的掌握。最后，我写了一个电子时钟程序，它不仅展示了我处理实时数据的能力，也让我学会了如何在屏幕上格式化显示信息。

综合这些任务，我学会了如何编写具有人机交互界面的复杂汇编应用程序，掌握了从键盘读取输入、在屏幕上显示结果以及处理实时时钟中断等关键技术。这些技能的提升为我未来在更高级的程序设计和系统开发中打下了坚实的基础。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验五 存储器扩展实验

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 5 月 24 日

成 绩

指导教师评语:
指导教师: 年 月 日

一、 实验要求

1. 使用两片 62256 扩充 RAM 空间为 00000H-0FFFFH。

2. 编制程序，将字符 1～8 循环写入 00000H-00FFFH 中，将 9 写入 01000H-02FFFH 中。将 01000H-02FFFH 中的内容搬移到 04000H-05FFFH 中。

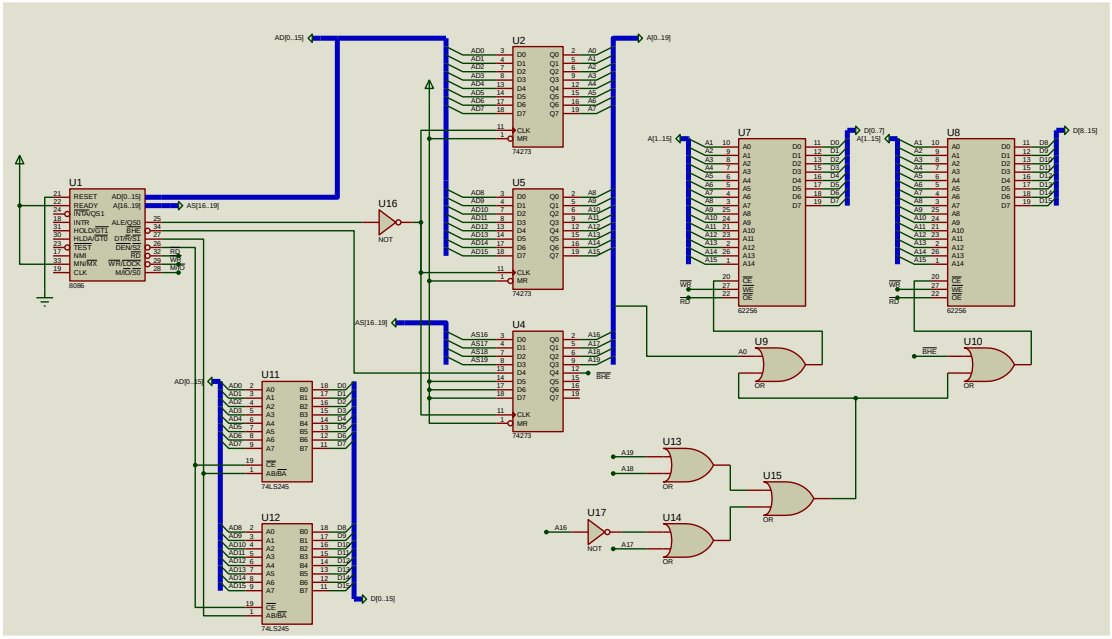
二、 实验目的

1. 学习 PROTEUS EDA 仿真工具的使用方法；

2. 学习 8086 最小系统中系统总线的形成方法；

3. 掌握内存扩充方法和外围接口方法，正确设计译码电路，按照要求的地址实现存储器芯片的连接。

三、 实验电路

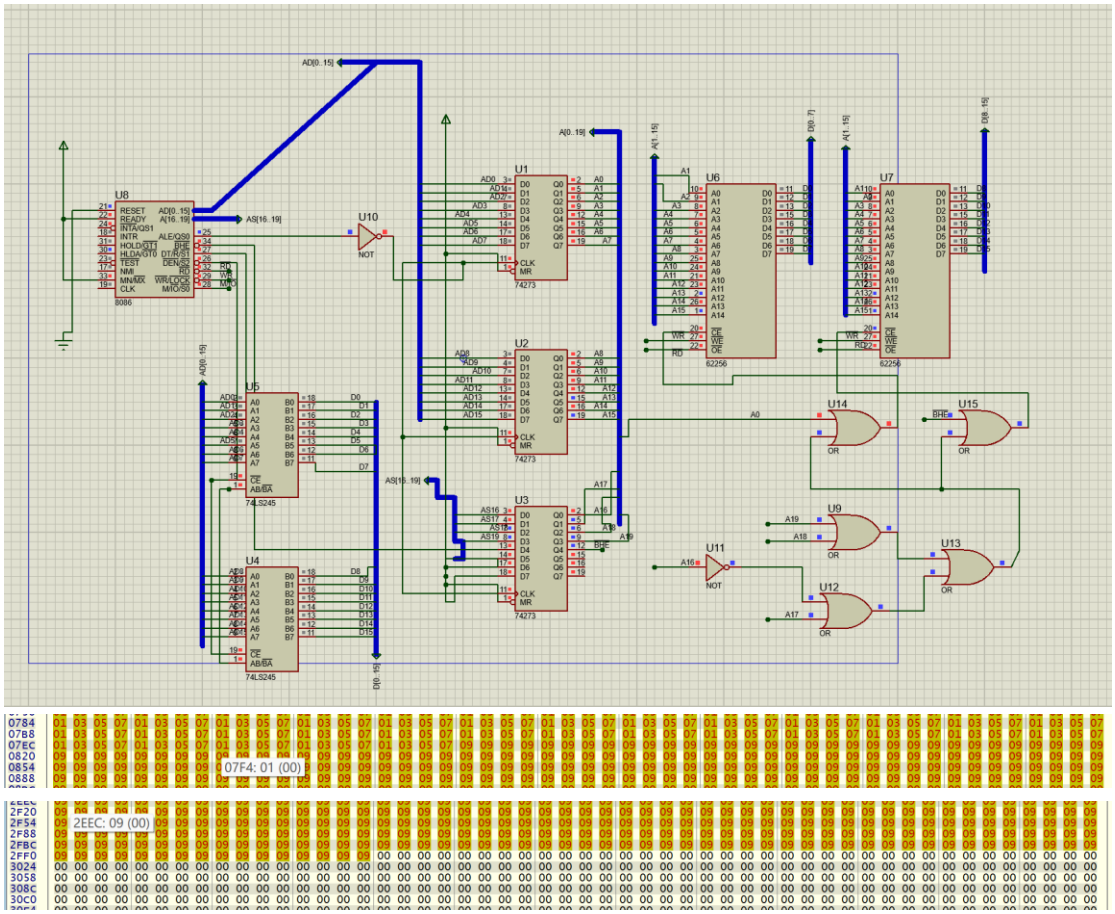


四、 实验代码及实验结果

```
CODE SEGMENT
    ASSUME CS:CODE
START:
    MOV AX,1000H
    MOV DS,AX ;设置数据段地址
    MOV AL,1
    MOV BX,0000H
    MOV CX,1000H ;循环次数
    MOV [BX],AL
    JMP M1
    ;1～8 循环写入 10000H-10FFFH
ZERO:
    MOV AL,1
```

```
M1:
      MOV [BX],AL
      INC BX
       INC AL
       CMP AL,8
       JA ZERO
       LOOP M1      ;循环次数 CX 的值
       ;将 9 写入 11000H-12FFFH
       MOV AL,09H
       MOV CX,2000H
       MOV BX,1000H ;偏移地址 1000H
M2:
       MOV [BX],AL
       INC BX
       LOOP M2
       ;将 11000H-12FFFH 内容移到 14000H-15FFFH
       MOV BX,1000H
       MOV CX,2000H
TRANS:
       MOV AL,[BX] ;读入
       ADD BX,3000H
       MOV [BX],AL
       SUB BX,3000H
       INC BX
       LOOP TRANS
       HLT
CODE ENDS
       END START
```

实验结果如下



五、 实验总结

本次实验中，我通过使用两片 62256 芯片成功扩充了 8086 系统的 RAM 空间，掌握了内存扩展的基本概念和实现方法。我对硬件电路设计有了初步了解，熟悉了 Proteus 的基本使用方法，掌握了如何将汇编程序载入到处理器中，从而完成了相应的功能。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验六 74LS273 扩展 I/O 口实验

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2022 年 6 月 1 日

成 绩

指导教师评语:

指导教师:

年 月 日

一、 实验要求

1、编写程序，以 74LS273 作为 I/O 口，控制 8 个单色 LED 灯。I/O 端口号为 168H。

2、设计一七段 LED 数码管显示电路，编写程序，实现在数码管从 0-9 的循环显示（间隔 1 秒）。（可使用 74LS47 作为 LED 的显示译码）思考：如果要求循环速度可调，即显示间隔可变，系统设计应做什么样的变化。

二、 实验目的

1、学习简单 I/O 接口的方法。

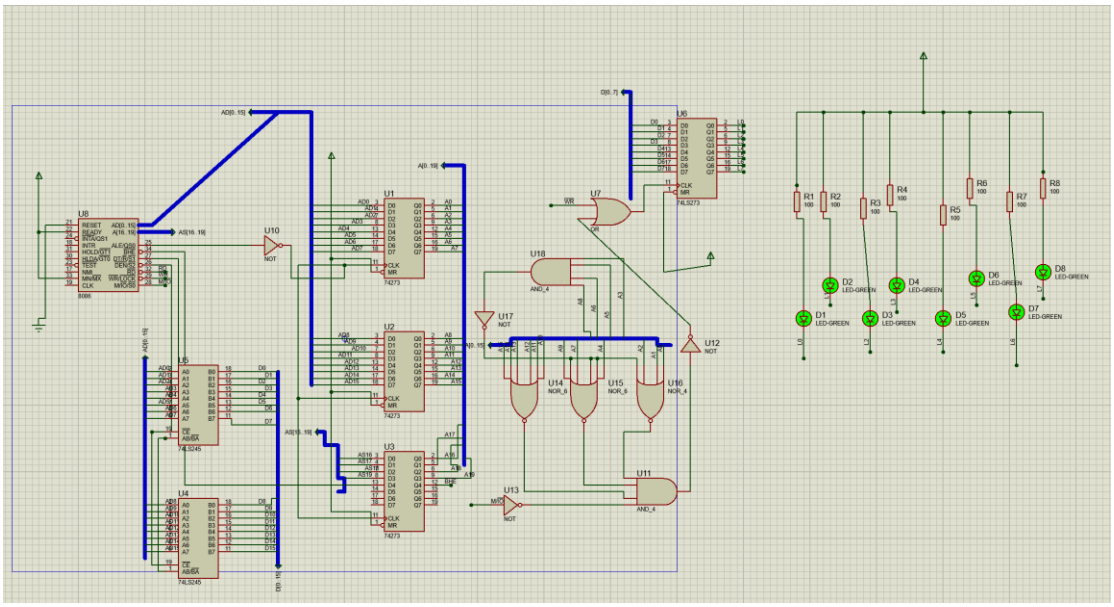
2、学习 74LS273 的使用方法。

3、学习编制数据向外设输出的程序。

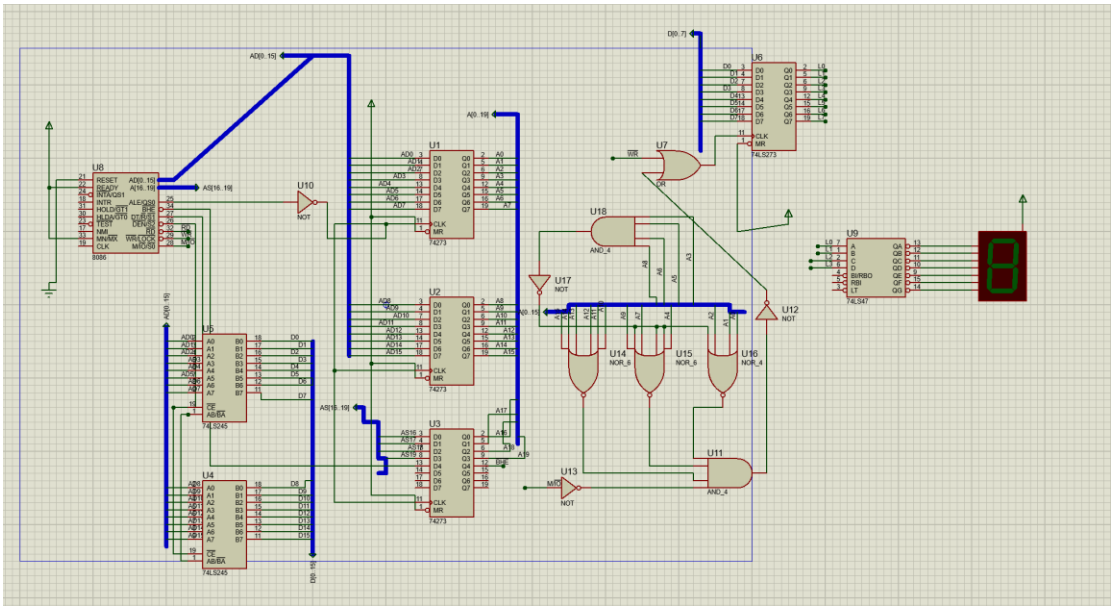
4、学习 LED 数码管的显示方法。

三、 实验电路

1.



2.



四、 实验代码及实验结果

1.

```
IO0 EQU 8000H
CODE SEGMENT
        ASSUME CS:CODE
START:    MOV DX,IO0
          MOV AL,0FEH
          OUT DX,AL
          MOV CX,0E000H
          LOOP $
          MOV AL,0FDH
          OUT DX,AL
          MOV CX,0E000H
          LOOP $
          MOV AL,0FBH
          OUT DX,AL
          MOV CX,0E000H
          LOOP $
          MOV AL,0F7H
          OUT DX,AL
          MOV CX,0E000H
          LOOP $
          MOV AL,0EFH
          OUT DX,AL
          MOV CX,0E000H
          LOOP $
          MOV AL,0DFH
```

```
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,0BFH
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,07FH
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        JMP START
ENDLESS:
        JMP ENDLESS
CODE    ENDS
        END START
```

2.

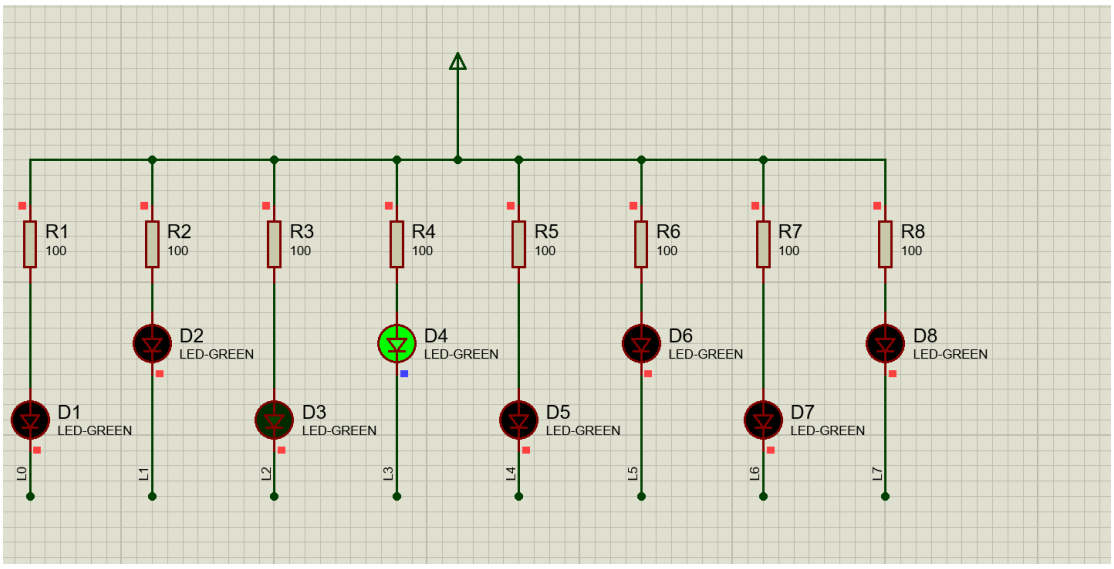
```
IO0 EQU 8000H
CODE SEGMENT
    ASSUME CS:CODE
START:  MOV DX,IO0
        MOV AL,00H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,01H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,02H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,03H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,04H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,05H
```

```
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,06H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,07H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,08H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $
        MOV AL,09H
        OUT DX,AL
        MOV CX,0E000H
        LOOP $

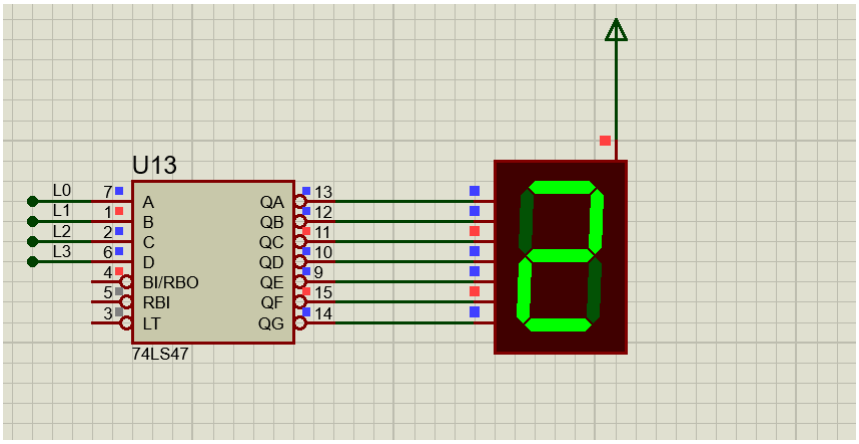
        JMP START
ENDLESS:
        JMP ENDLESS
CODE      ENDS
        END START
```

实验结果：

1、



2、



五、 实验总结

在本次实验中，我通过编写程序控制 74LS273 芯片，实现了对 8 个单色 LED 灯的控制。通过直接操作 I/O 端口 168H，我学习了如何将数据输出到硬件设备，并观察了实际的硬件响应。

进一步地，我实现了一个七段 LED 数码管的显示电路，并通过程序实现了 0 至 9 数字的循环显示，加深了我对 74LS47 译码器的理解，锻炼了我的编程能力。

总体而言，本次实验不仅让我学习了 74LS273 和 74LS47 芯片的使用方法，还增强了编制数据向外设输出程序的能力，为未来在更复杂的硬件编程和系统设计方面奠定了基础。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验七 基于 8255 的开关状态显示实验

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 6 月 8 日

成 绩

指导教师评语:

指导教师:

年 月 日

一、 实验要求

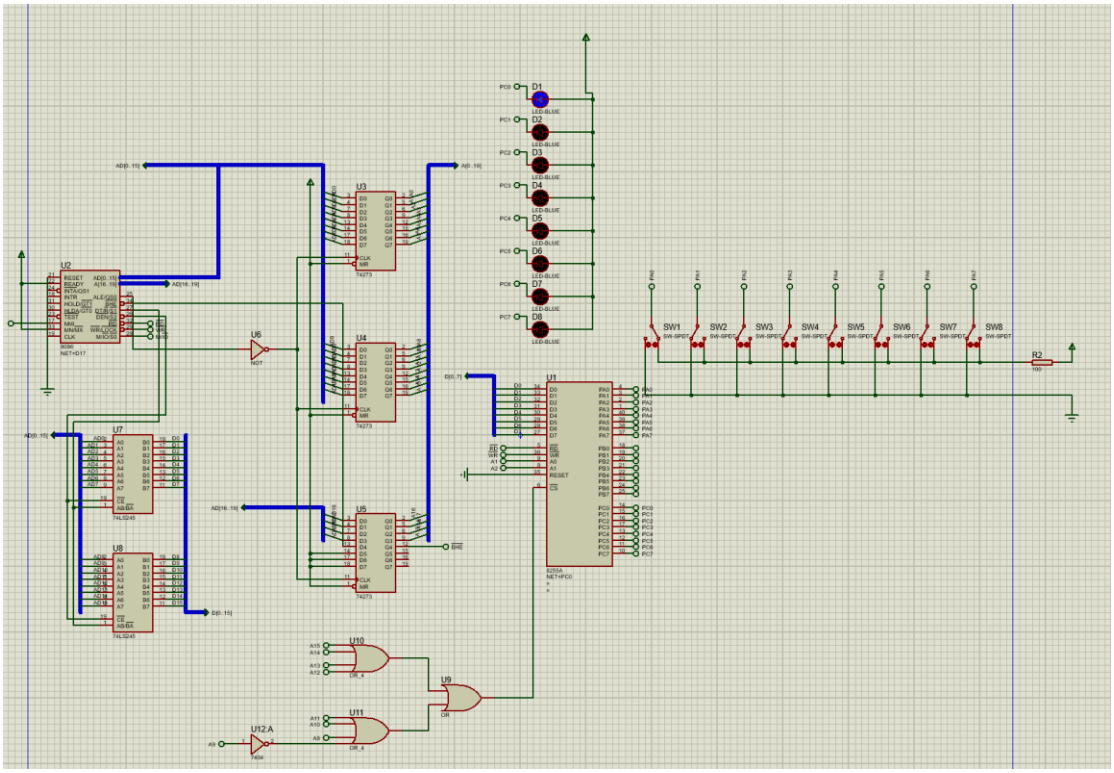
1. 编写程序，设定 8255 的 PA 口为开关量输入，PC 口为开关量输出，要求能随时将 PA 口的开关状态通过 PC 口的发光二极管显示出来。
2. 用 8255A 实现一电子秒表，计时范围 0-99.99 秒，应具有“START”，“STOP”，“CLEAR”三个按键，分别完成开始，停止，清零功能。

二、 实验目的

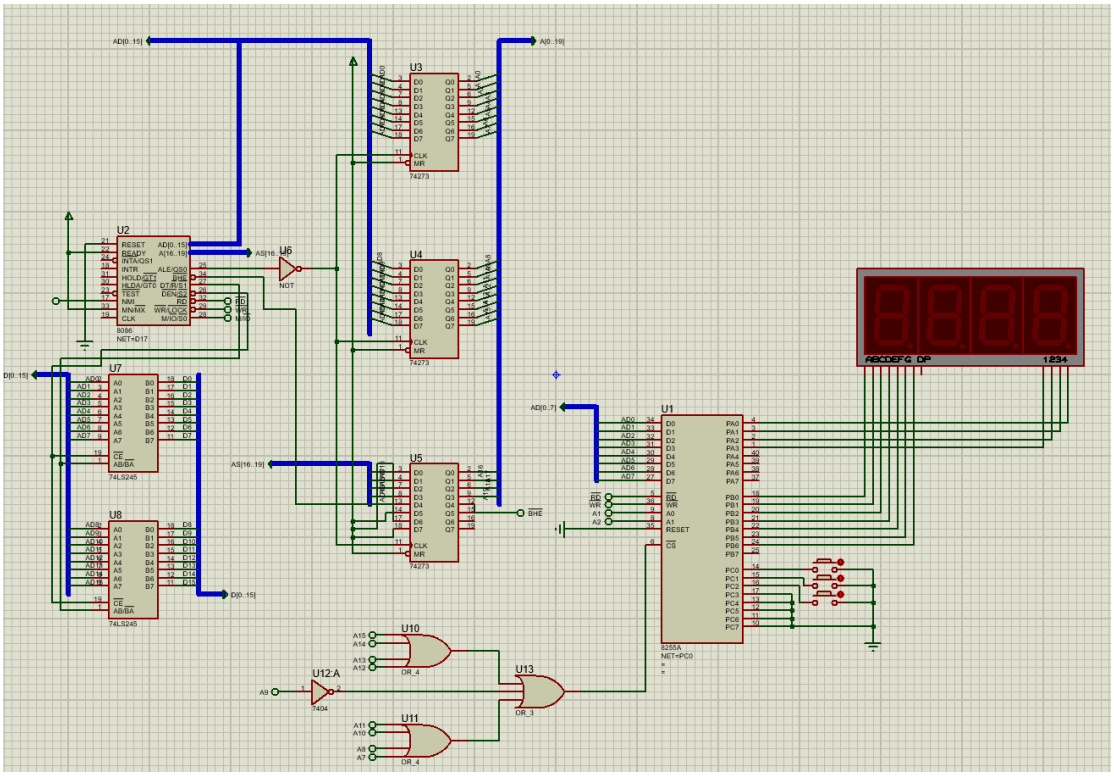
学习使用 8255 各个口的不同工作方式。

三、 实验电路

1.



2.



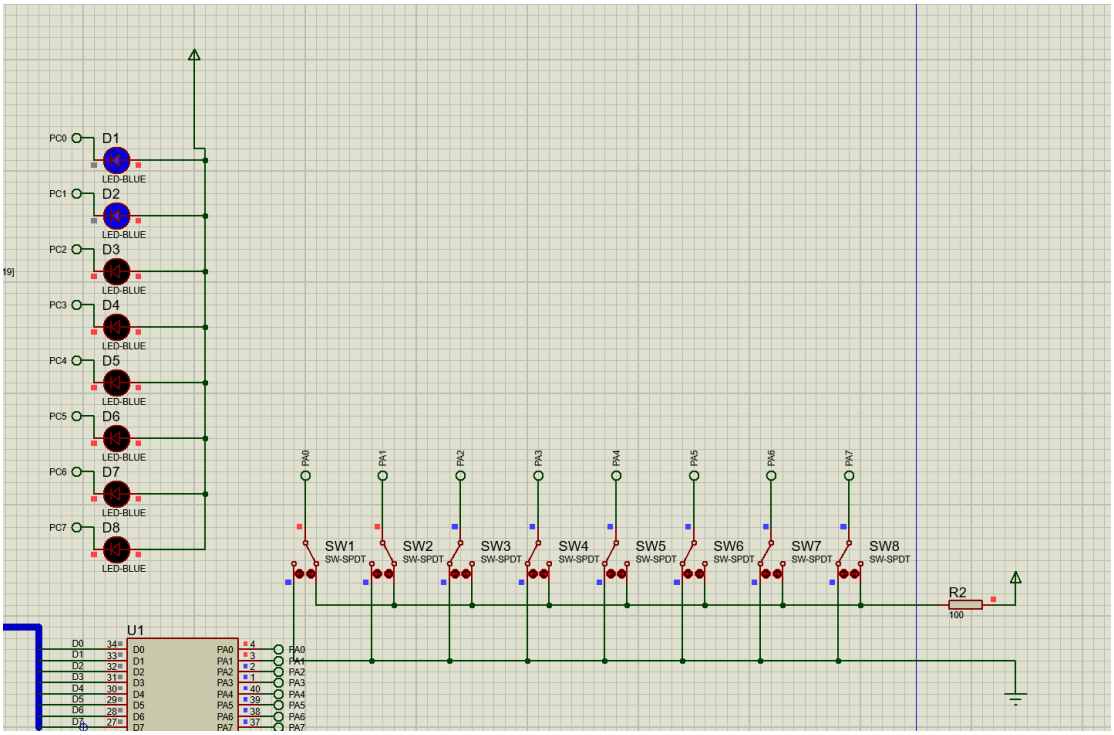
四、 实验代码及实验结果

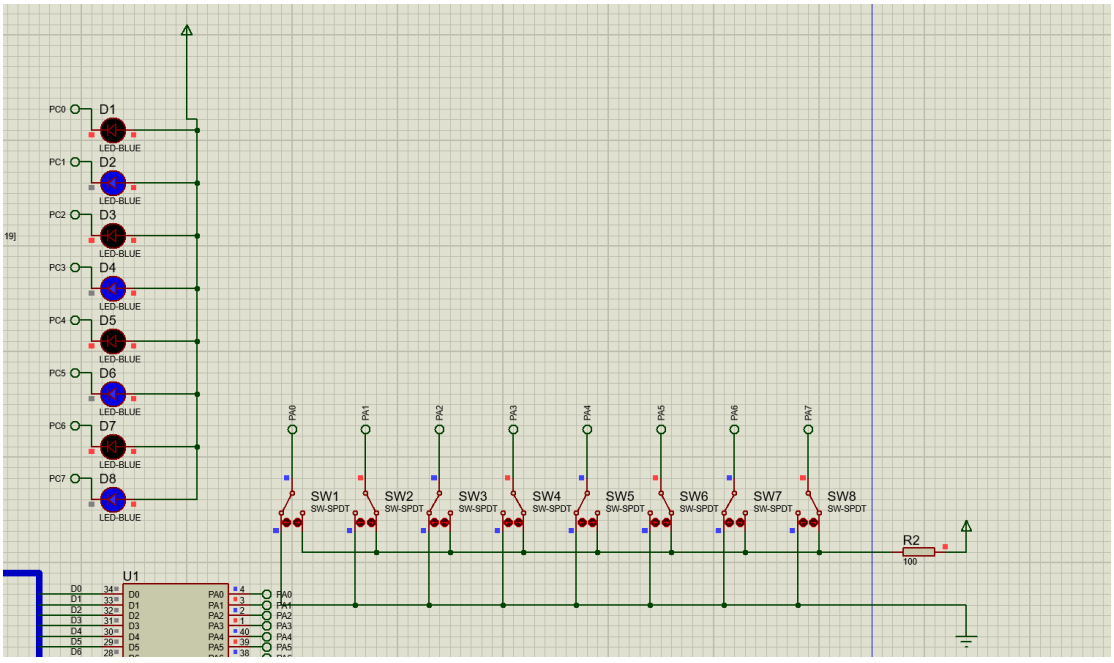
1.

实验代码：

```
CODE SEGMENT
    ASSUME CS:CODE
START:
    MOV AL,90H ;控制 A 输入 C 输出
    MOV DX,206H
    OUT DX,AL
RUN:
    MOV AL, 0H
    MOV DX, 200H
    IN AL, DX ;端口 A 读入
    NOT AL
    MOV DX, 204H
    OUT DX, AL ;端口 C 输出
    JMP RUN
CODE ENDS
END START
```

实验结果：





2.

实验代码：

```
DATA SEGMENT
    NUMBER DB 40H,79H,24H,30H,19H,12H,02H,78H,00H,10H
    MIN1 DB 0
    MIN2 DB 0
    SEC1 DB 0
    SEC2 DB 0
    CIR DW 800
DATA ENDS
```

```
CODE SEGMENT
    ASSUME CS:CODE,DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AL, 89H
    MOV DX, 206H   ;初始化
    OUT DX, AL     ;A,B 输出，C 输入
    MOV BX, 0H
    LEA SI, NUMBER

OUTPUT:
    MOV AL, 00000001B
    MOV DX, 200H      ;选中第 4 个数码管
    OUT DX, AL
    MOV BL, SEC2      ;输出 SEC2
    MOV AL, [SI+BX]
    MOV DX, 202H
    OUT DX, AL
    MOV CX, CIR
S4:
    LOOP S4

    MOV AL, 00000010B
    MOV DX, 200H      ;选中第 3 个数码管
    OUT DX, AL
    MOV BL, SEC1
    MOV AL, [SI+BX]
    MOV DX, 202H
    OUT DX, AL
    MOV CX, CIR
S3:
    LOOP S3

    MOV AL, 00000100B
    MOV DX, 200H      ;选中第 2 个数码管
    OUT DX, AL
    MOV BL, MIN2
    MOV AL, [SI+BX]
    MOV DX, 202H
    OUT DX, AL
    MOV CX, CIR
S2:
```

```
        LOOP S2

        MOV AL, 00001000B
        MOV DX, 200H        ;选中第 3 个数码管
        OUT DX, AL
        MOV BL, MIN1
        MOV AL, [SI+BX]
        MOV DX, 202H
        OUT DX, AL
        MOV CX, CIR
S1:
        LOOP S1
        ;四个数码管显示结束，判断 C 端口状态
        MOV DX, 204H
        IN AL, DX
        CMP AL, 06H
        JE STOP       ;STOP 暂停计时
        CMP AL, 03H
        JE TAIL1      ;CLEAR 清零计时

        ;没有按按键或者按了 STRAT，增加秒数并判断是否进位
        INC SEC2
        CMP SEC2, 10
        JNZ OUTPUT

        ;SEC2 = 10,进位
        MOV SEC2, 0H
        INC SEC1
        CMP SEC1, 10
        JNZ OUTPUT

        ;SEC1 = 10,进位
        MOV SEC1, 0H
        INC MIN2
        CMP MIN2, 10
        JNZ OUTPUT

        ;MIN2 = 10,  进位
        MOV MIN2, 0H
        INC MIN1
        CMP MIN1, 10
        JNZ OUTPUT

        ;MIN1 = 10,清零
```

```
        MOV MIN1, 0H
        JMP OUTPUT

STOP:
        MOV AL, 00000001B
        MOV DX, 200H        ;选中第 4 个数码管
        OUT DX, AL
        MOV BL, SEC2
        MOV AL, [SI+BX]
        MOV DX, 202H
        OUT DX, AL
        MOV CX, CIR
S8:
        LOOP S8

        MOV AL, 00000010B
        MOV DX, 200H        ;选中第 3 个数码管
        OUT DX, AL
        MOV BL, SEC1
        MOV AL, [SI+BX]
        MOV DX, 202H
        OUT DX, AL
        MOV CX, CIR
S7:
        LOOP S7

        MOV AL, 00000100B
        MOV DX, 200H        ;选中第 2 个数码管
        OUT DX, AL
        MOV BL, MIN2
        MOV AL, [SI+BX]
        MOV DX, 202H
        OUT DX, AL
        MOV CX, CIR
S6:
        LOOP S6

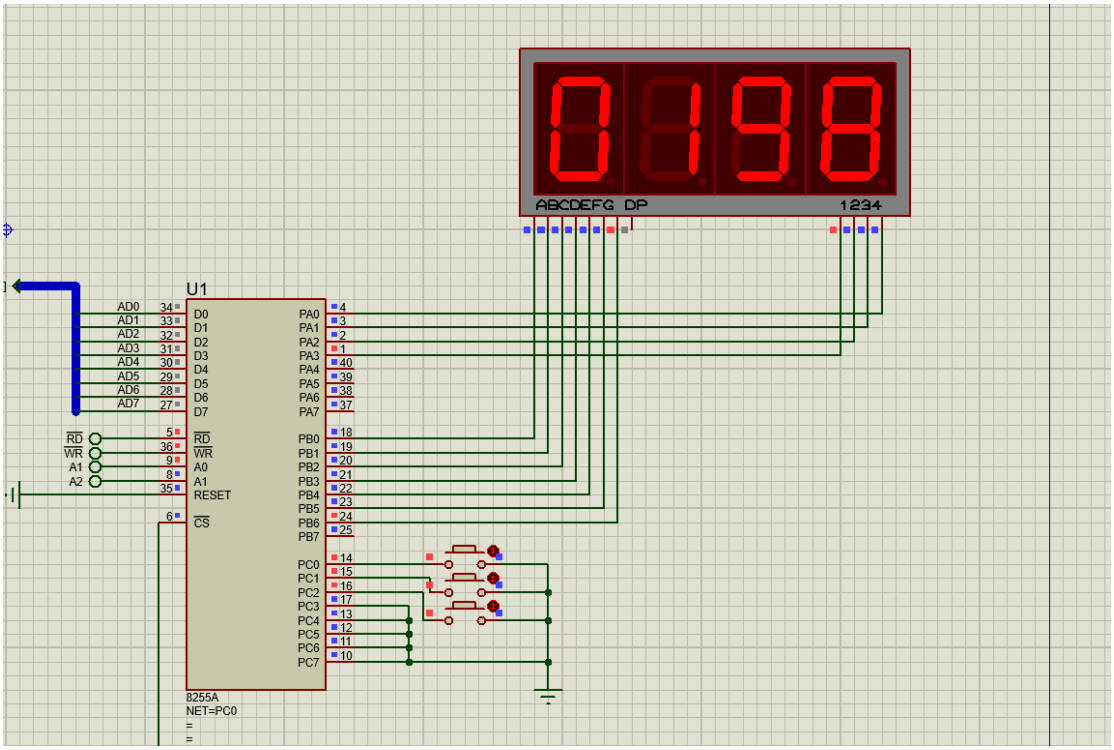
        MOV AL, 00001000B
        MOV DX, 200H        ;选中第 1 个数码管
        OUT DX, AL
        MOV BL, MIN1
        MOV AL, [SI+BX]
        MOV DX, 202H
        OUT DX, AL
```

```
        MOV CX, CIR
    S5:
        LOOP S5
        ; 判断是继续计时、暂停还是清零
        MOV DX, 204H
        IN AL, DX
        CMP AL, 05H ;START 按键按下
        JE   OUTPUT
        CMP AL, 03H ;CLEAR 按键按下
        JE TAIL2
        JMP STOP

    TAIL1:
        CALL ZERO
        JMP OUTPUT
    TAIL2:
        CALL ZERO
        JMP STOP
    ZERO PROC
        MOV SEC2, 0H
        MOV SEC1, 0H
        MOV MIN1, 0H
        MOV MIN2, 0H
        RET
    ZERO ENDP
    CODE ENDS
END START
```

实验结果：

Stop 后的结果：



五、 实验总结

本次实验通过编程 8255 芯片，我掌握了其不同工作模式的配置与应用。

实验中，我首先实现了将 PA 口的输入状态通过 PC 口的 LED 实时显示，加深了对 I/O 端口操作的理解。

随后，设计了一个电子秒表，通过“START”，“STOP”，“CLEAR”按键控制计时，实现了 0 至 99.99 秒的精确计时。这一过程提升了我的硬件接口编程能力，并加深了对并行接口芯片工作机制的认识。

通过本次实验，我更加熟悉了微机系统中的接口技术，为后续复杂系统设计打下了基础。

西安电子科技大学

微机系统综合实验 课程实验报告

实验名称 实验八 点阵 LED 显示设计实验

网络与信息安全学院 2218039 班

姓名 廖鑫洋 学号 22009201118

同作者 无

实验日期 2024 年 6 月 15 日

成 绩

指导教师评语:

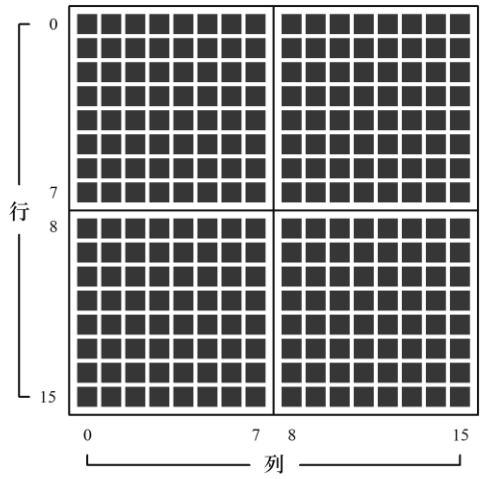
指导教师:

年 月 日

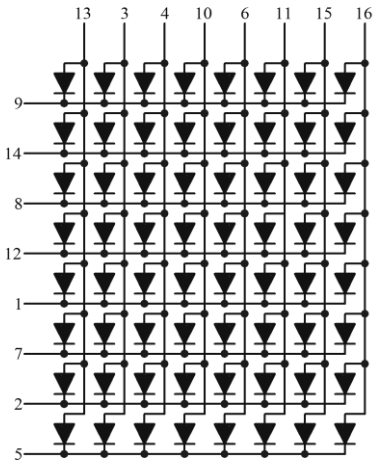
一、 实验要求

1. 实现 8×8 LED 点阵实验，要求在 8×8 的 LED 上循环显示 1，2，3 这三个数字。

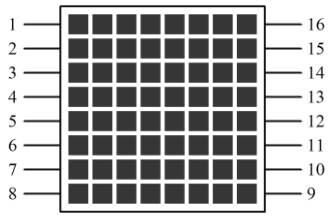
2. 实验系统中的 16×16 LED 点阵由四块 8×8 LED 点阵组成，控制点阵向上（或左右）游动显示“西安电子科技大学欢迎您！-姓名”。其中，实验系统中的 16×16 LED 点阵由四块 8×8 LED 点阵组成，如图所示，8×8 点阵内部结构图如下。由图可知，当行为“0”，列为“1”，则对应行、列上的 LED 点亮。



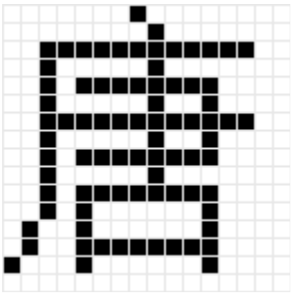
16×16 点阵示意图



点阵内部结构图



点阵外部引脚图

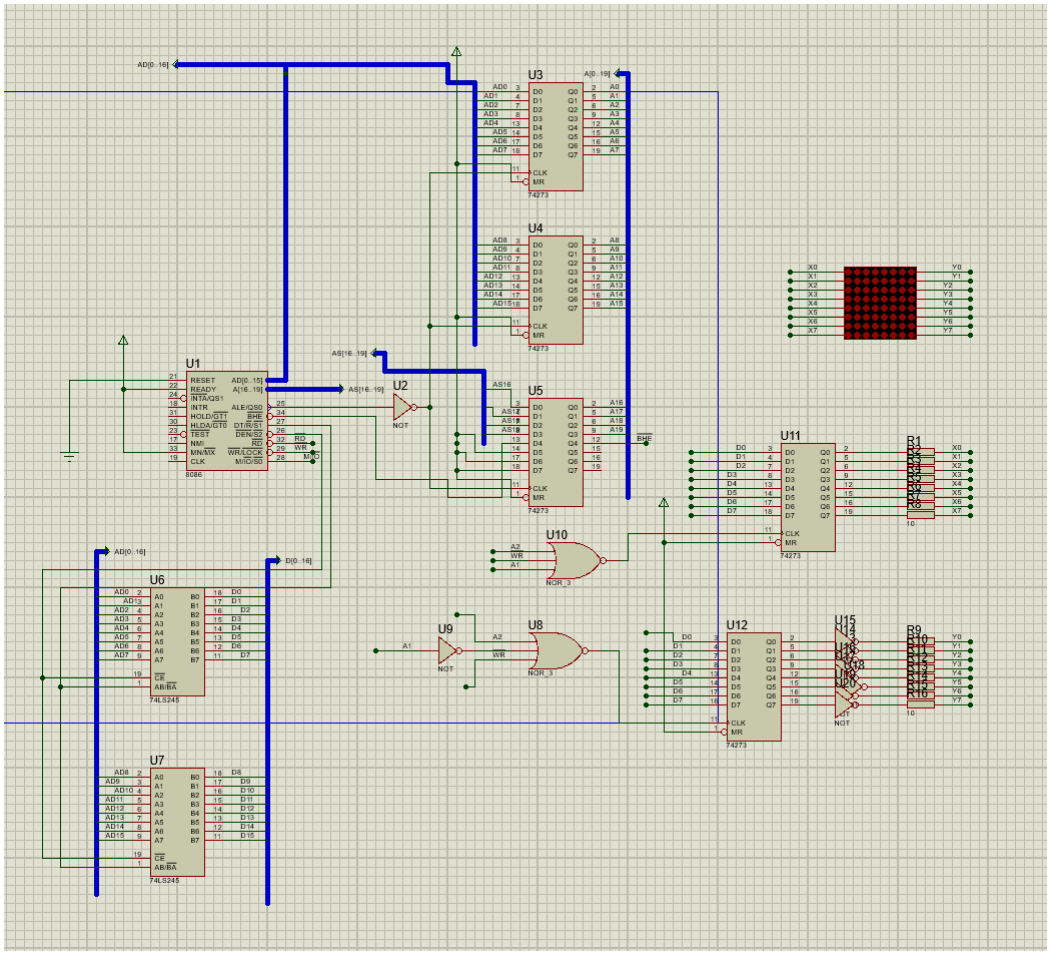


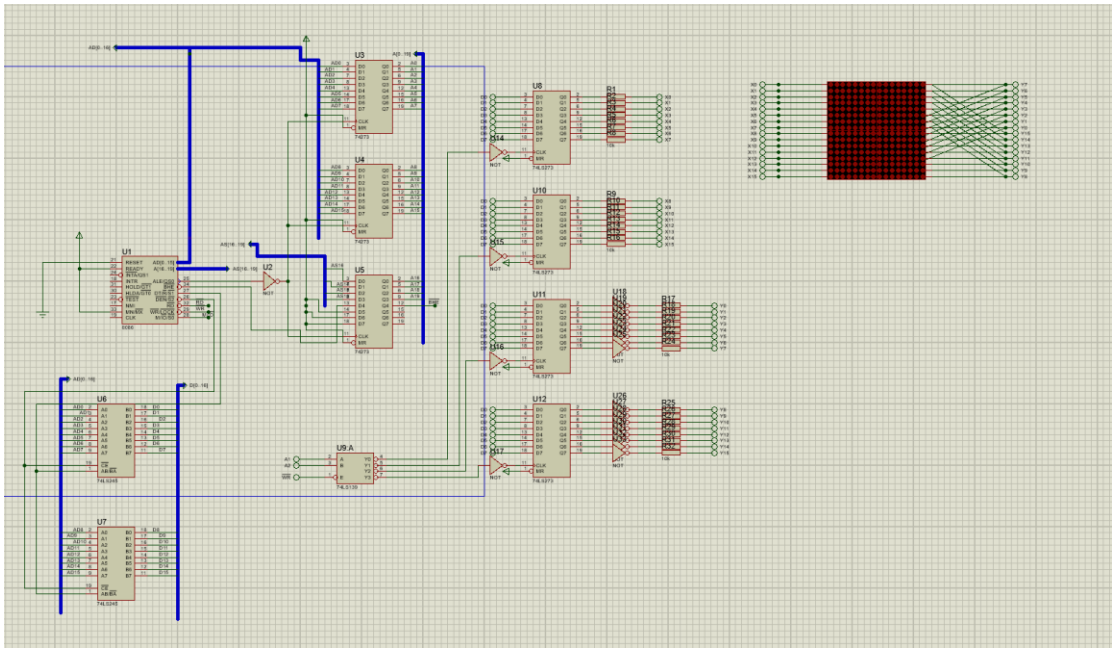
汉字显示示例

二、 实验目的

1. 了解 LED 点阵的基本结构。
2. 学习 LED 点阵扫描显示程序的设计方法。

三、 实验电路



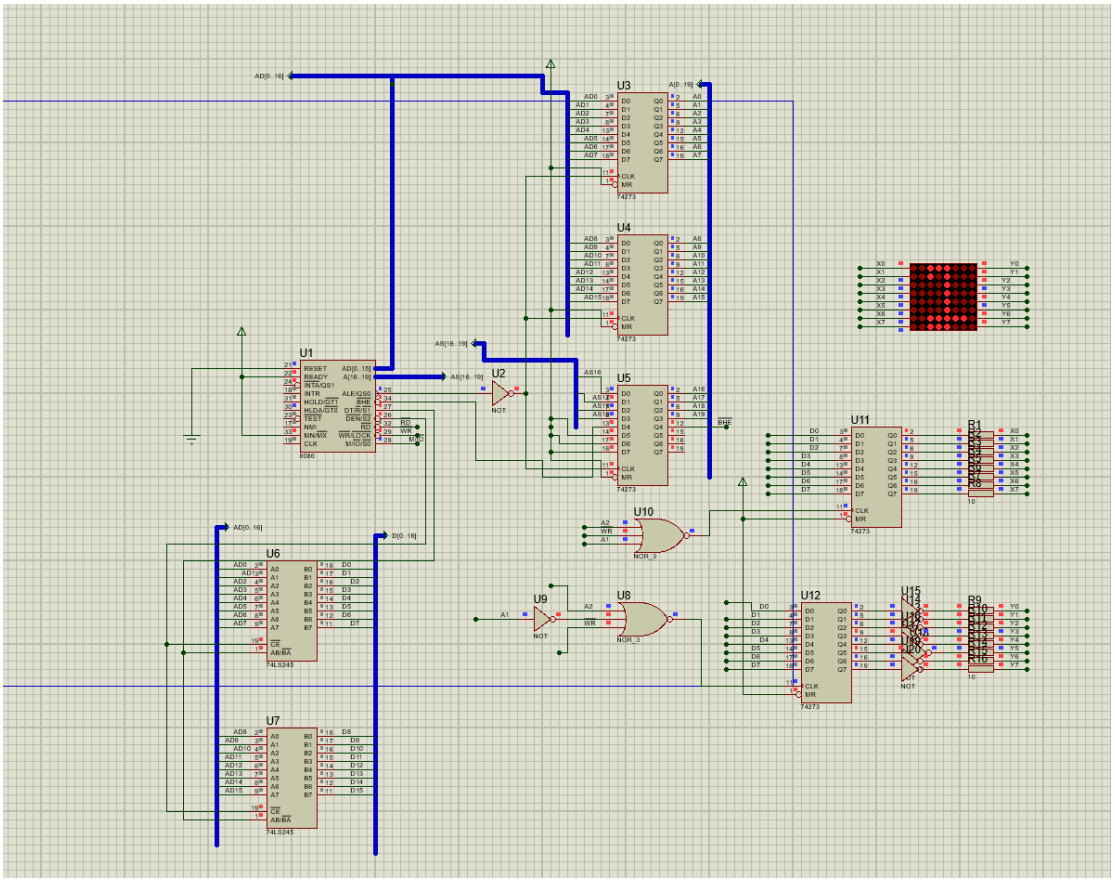


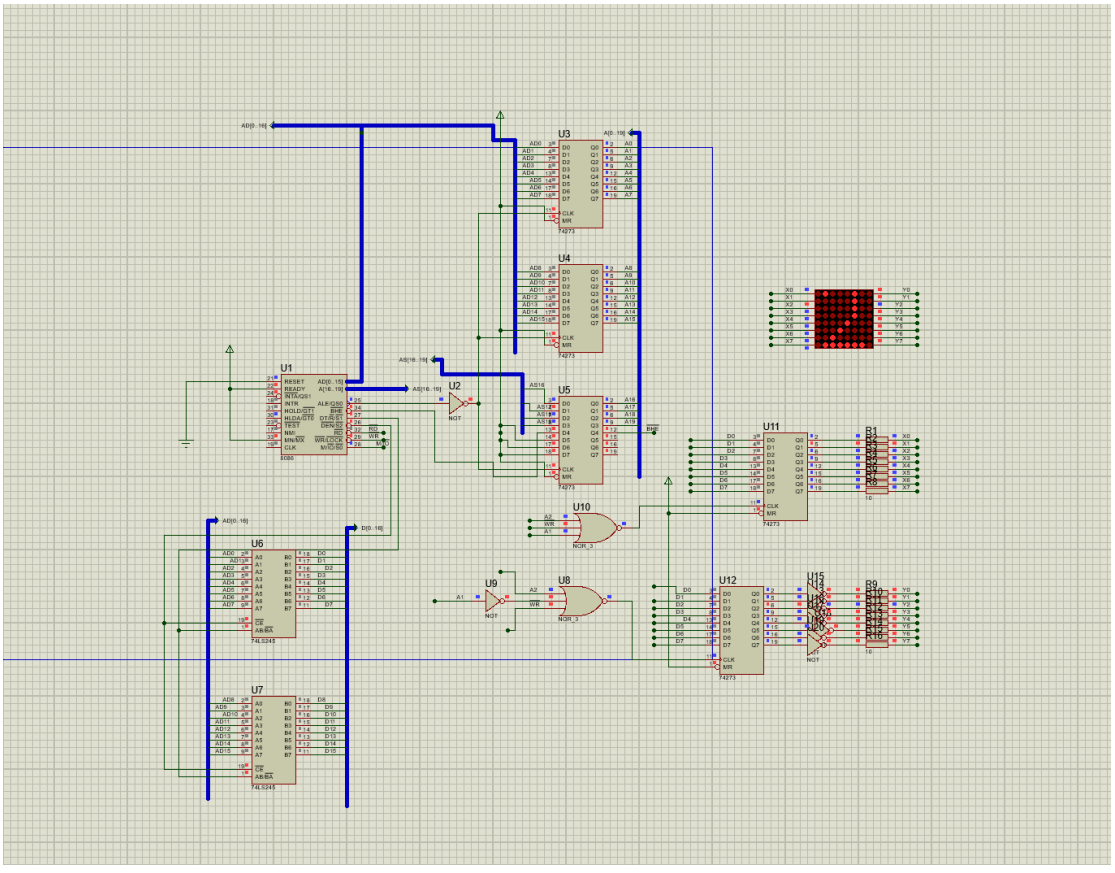
四、 实验代码及实验结果

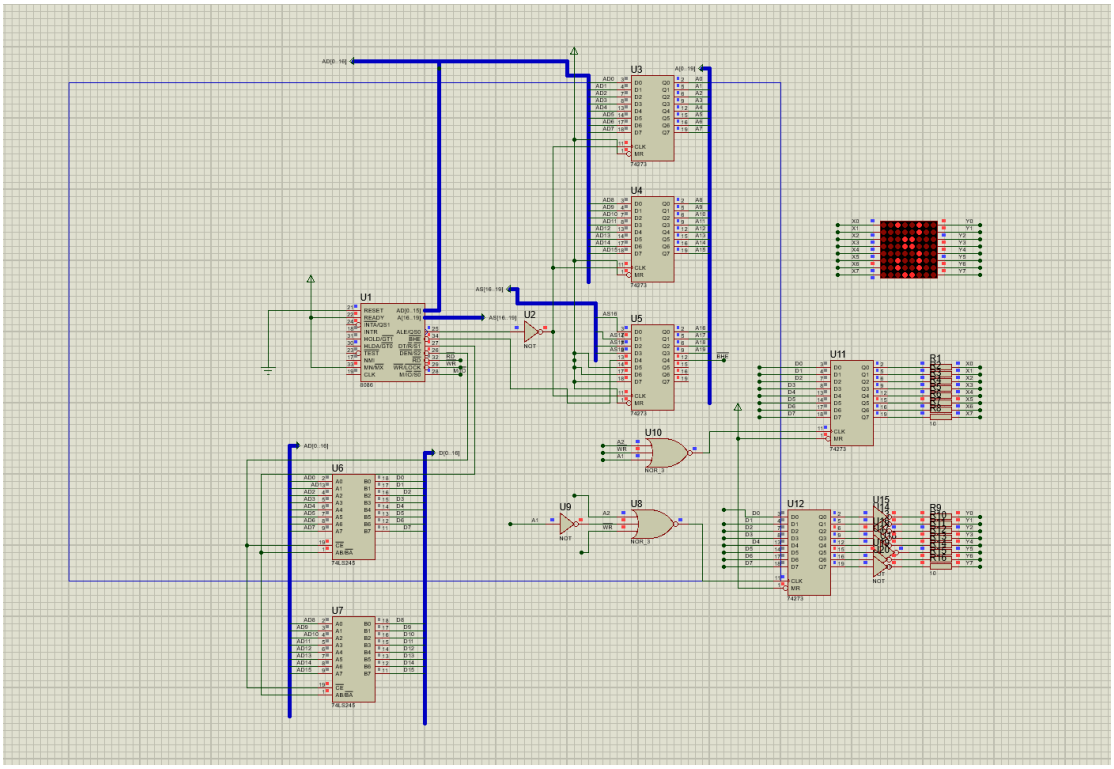
1、 实验一实验代码

```
DATA SEGMENT
    NUMS:
    DB 08H,18H,28H,08H,08H,08H,08H,3EH ;数字 1
    DB 38H,44H,04H,04H,08H,10H,20H,7EH ;数字 2
    DB 18H,24H,04H,18H,18H,04H,24H,18H ;数字 3
DATA ENDS
CODE SEGMENT
    ASSUME CS:CODE,DS:DATA
START:
    MOV AX,DATA
    MOV DS,AX
    MOV AH,01H
    MOV BX,00H
RUN:
    MOV DX,OFFSET NUMS
    MOV SI,DX
    ADD SI,BX
    MOV CX,08H
WAITE:
    MOV AL,0H
    OUT 00H,AL
    MOV AL,[SI]
    OUT 02H,AL
    MOV AL,AH
    OUT 00H,AL
    ROL AH,1
    PUSH CX
    MOV CX,700H
    LOOP $
    POP CX
    INC SI
    LOOP WAITE
    INC BX
    CMP BX,24
    JA RESET
CONTINUE:
    JMP RUN
RESET:
    MOV BX,00H
    JMP CONTINUE
CODE ENDS
END START
```

2、 实验一实验结果


U1
U2
U3
U4
U5
U6
U7
U8
U9
U10
U11
U12
U15
R1
R9
8086
74273
74LS245
NOT
NOR_3


U1
U2
U3
U4
U5
U6
U7
U8
U9
U10
U11
U12
U15
R1
R9
8086
74273
74LS245
NOT
NOR_3



3、 实验二实验代码

```
DATA SEGMENT
    LEFT:;左侧 1*2 块屏幕，从低位到高位,每个字节是半行
    DB 0H, 0FFH, 20H, 20H, 20H, 0FCH,24H, 24H, 24H, 24H, 14H, 0CH, 04H, 4H,0FCH, 4H       ;西
    DB 40H, 80H, 0FCH, 04H, 42H, 40H, 40H,0FFH, 20H, 10H, 18H, 60H, 80H,40H, 30H,0EH       ;安
    DB 80H, 80H,  80H,0FCH, 84H, 84H, 84H,0FCH, 84H, 84H, 84H,0FCH, 84H,80H, 80H,00H       ;电
    DB 00H,0FEH,  00H, 00H, 00H, 80H, 80H,0FFH, 80H, 80H, 80H, 80H, 80H,80H,0A0H,40H       ;子
    DB 10H,0B8H,  0FH, 08H, 08H,0BFH,  8H,01CH, 2CH, 0AH,0CAH, 9H,  8H, 8H,  8H, 8H     ;科
    DB 8H,  8H,  8H,0C8H, 3FH,  8H,  8H,0A8H, 18H, 0CH, 0BH, 08H, 08H, 08H, 08H,064H     ;技
    DB 80H, 80H, 80H, 80H, 80H,0FFH, 80H, 80H, 40H, 40H, 20H, 20H, 10H, 8H,  4H,  3H     ;大
    DB 44H, 88H, 88H,  0H,0FEH,  2H,  1H,0F8H,  0H, 80H,0FFH, 80H, 80H, 80H,0A0H, 40H     ;学
    DB 0H,    0H, 3FH, 20H,0A0H, 92H, 54H, 28H,  8H, 14H, 24H,0A2H, 81H, 40H, 20H, 10H   ;欢
    DB 0H,    4H,0C8H, 48H, 40H, 40H, 4FH, 48H, 48H, 48H,0C8H, 48H,  8H, 14H,0E2H,  0H     ;迎
    DB 90H, 90H, 88H, 4CH, 2AH, 99H, 88H, 48H, 28H, 88H,  8H, 40H, 8AH,
```

8AH, 9H,0F0H ;您

DB 00H, 0FCH, 04H, 0F4H, 94H, 0A4H, 94H, 84H, 64H, 1CH, 0C4H, 34H, 88H, 72H, 81H, 70H ;廖

DB 80H, 60H, 18H, 0F7H, 80H, 0F8H, 90H, 0FCH, 08H, 14H, 3EH, 49H, 3EH, 08H, 2AH, 7FH ;鑫

DB 40H, 84H, 88H, 08H, 0E1H, 02H, 02H, 0E8H, 08H, 04H, 0E7H, 04H, 04H, 04H, 04H, 00H ;洋

RIGHT:;右侧 1*2 块屏幕

DB 00H, 07FH,02H,02H,02H,1FH,12H,12H,12H,12H,1CH,10H,10H,10H,1FH,10H

DB 00H, 00H,3FH,20H,10H,00H,00H,7FH,04H,04H,02H,02H,01H,06H,08H,10H

DB 00H, 00H,00H,01FH,10H,10H,10H,01FH,10H,10H,10H,01FH,50H,40H,40H,7FH

DB 00H, 01FH,08H,04H,02H,01H,00H,07FH,00H,00H,00H,00H,00H,00H,00H,00H

DB 08H, 08H,09H,09H,08H,008H,9H,09H,08H,78H,0FH, 08H,08H,08H,08H,08H

DB 04H, 04H,4H,07FH,04H,04H,04H,3FH,21H,11H,12H,0AH,4H,0AH,11H,60H

DB 00H, 00H,00H,00H,00H,07FH,00H,00H,01H,01H,02H,02H, 4H,8H,10H,60H

DB 10H, 10H,08H,4H,07FH,40H,20H,7H,2H,01H,07FH,00H,00H,00H,00H, 0H

DB 01H, 01H, 1H,3FH, 20H,10H,02H, 2H,2H, 5H, 5H, 08H, 8H,10H,20H,40H

DB 00H, 01H, 3CH,24H,24H,24H,24H,24H,24H,2DH,14H, 04H,4H, 4H,07FH,0H

DB 00H, 00H,3FH,20H,12H,0AH,12H,22H,22H,2H, 1H, 0H,20H,48H,48H,0FH

DB 01H, 7FH, 00H, 3EH, 24H, 28H, 25H, 06H, 18H, 61H, 04H, 02H, 11H, 0CH, 03H, 00H

DB 00H, 03H, 0CH, 77H, 00H, 0FH, 04H, 1FH, 08H, 14H, 3EH, 49H, 3EH, 08H, 2AH, 7FH

DB 10H, 10H, 08H, 00H, 3FH, 02H, 02H, 1FH, 04H, 04H, 7FH, 04H, 04H, 04H, 04H, 00H

```
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE,DS:DATA
START:
    MOV AX,DATA
    MOV DS,AX
    MOV AH,01H
    MOV BX,00H ;滚动行数(移向下一字节代表从下一行开始打印)
RUN:
    MOV DX,OFFSET LEFT
    MOV SI,DX
    ADD SI,BX     ;计算当前行数（本轮从该行开始打印）
    MOV DX,OFFSET RIGHT
    MOV DI,DX
    ADD DI,BX     ;计算当前行数（本轮从该行开始打印）
    MOV CX,08H;共 8 行
WRITE1:     ;在上面两块屏幕上输出
    MOV AL,0H
    OUT 0H,AL;清屏操作
    MOV AL,[SI];左边内容
    OUT 4H,AL
    MOV AL,[DI];右边内容
    OUT 6H,AL
    MOV AL,AH     ;设置目标行
    OUT 0H,AL
    ROL AH,1;目标行下移
    INC SI;指向下一行
    INC DI
    PUSH CX
    MOV CX,0500H ;延迟
    LOOP $
    POP CX
    LOOP WRITE1

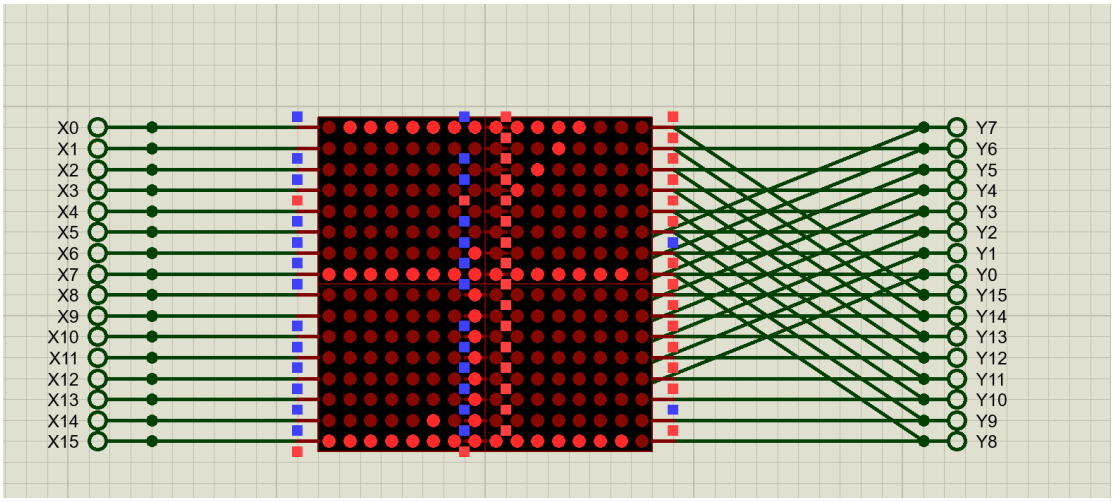
    MOV AL,0H
    OUT 0H,AL
    MOV CX,08H
WRITE2:;在下面两块屏幕上输出
    MOV AL,0H
    OUT 2H,AL ;清屏操作
    MOV AL,[SI] ;左边内容
    OUT 4H,AL
    MOV AL,[DI] ;右边内容
```

```
        OUT 6H,AL
        MOV AL,AH     ;设置目标行
        OUT 2H,AL
        ROL AH,1    ;目标行下移
        INC SI
        INC DI
        PUSH CX
        MOV CX,0500H
        LOOP $
        POP CX
        LOOP WRITE2

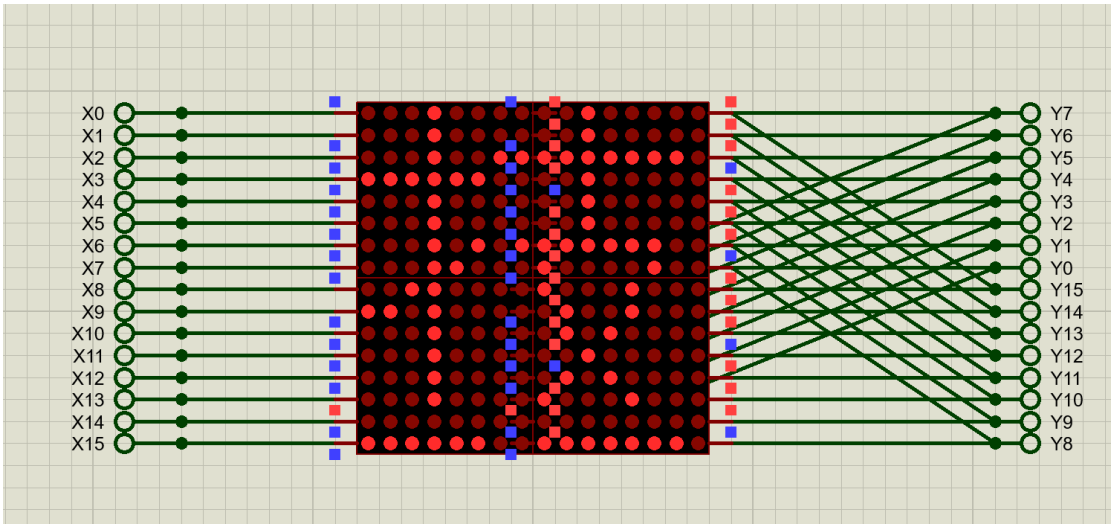
        INC BX
        CMP BX,224 ;14 个字，向上滚 14*16 行
        JA RESET
CONTINUE:
        JMP RUN
RESET: ;滚动值清零
        MOV BX,0
        JMP CONTINUE
CODE ENDS
END START
```

4、 实验二实验结果

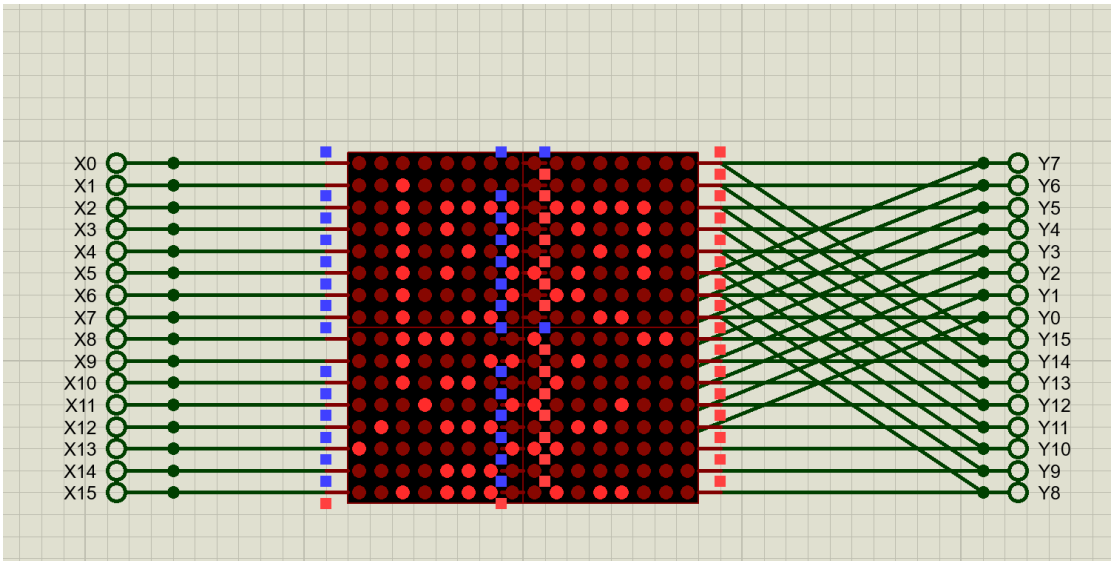
“子”：



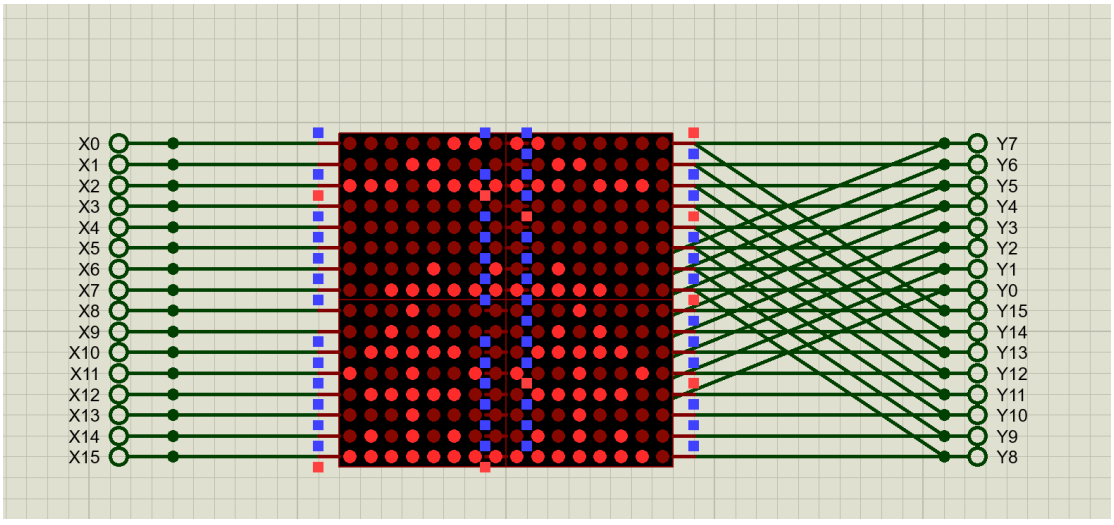
“技”：


X0
X1
X2
X3
X4
X5
X6
X7
X8
X9
X10
X11
X12
X13
X14
X15
Y7
Y6
Y5
Y4
Y3
Y2
Y1
Y0
Y15
Y14
Y13
Y12
Y11
Y10
Y9
Y8

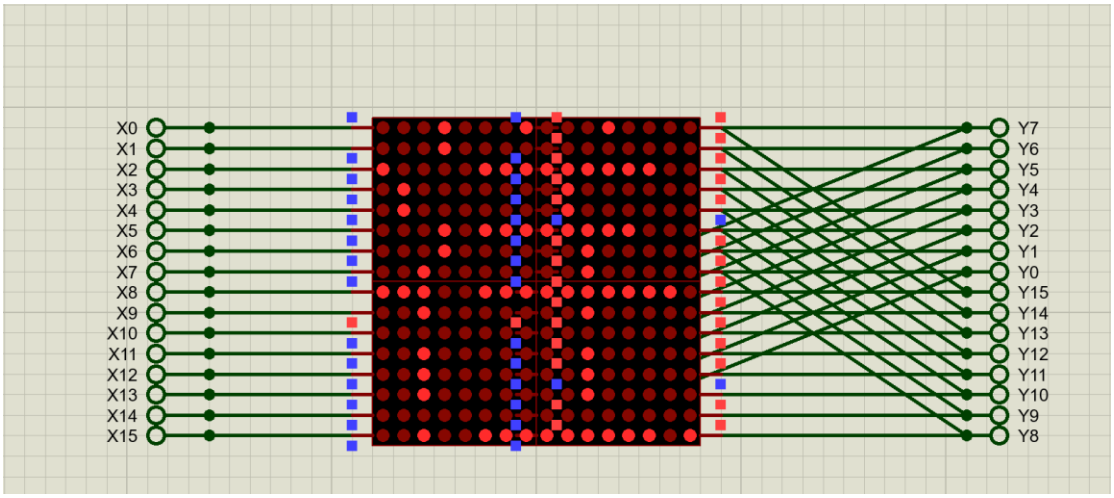
“廖”：


X0
X1
X2
X3
X4
X5
X6
X7
X8
X9
X10
X11
X12
X13
X14
X15
Y7
Y6
Y5
Y4
Y3
Y2
Y1
Y0
Y15
Y14
Y13
Y12
Y11
Y10
Y9
Y8

“鑫”：


X0
X1
X2
X3
X4
X5
X6
X7
X8
X9
X10
X11
X12
X13
X14
X15
Y7
Y6
Y5
Y4
Y3
Y2
Y1
Y0
Y15
Y14
Y13
Y12
Y11
Y10
Y9
Y8

“洋”：



五、 实验总结

通过本次 LED 点阵实验，我对 LED 点阵的基本结构有了深入的了解，并且掌握了 LED 点阵扫描显示数字、文字的设计方法。

实验中，我首先实现了在 8×8 LED 点阵上循环显示数字 1、2、3 的功能，这让我学会了如何控制单个点阵的显示内容。

进一步地，我实现了在 16×16 LED 点阵上滚动显示“西安电子科技大学欢迎您！廖鑫洋”的文字。这个过程加深了我对多位点阵组合显示的理解，并且锻炼了我设计复杂显示模式的能力。

在实验过程中，遇到了不少挑战，比如如何优化代码以减少延迟，如何调整时序以实现平滑的滚动效果，以及如何处理多位点阵之间的同步显示。通过不断尝试和调试，我逐步克服了这些问题，并最终完成了实验要求。

总之，这次实验让我对 LED 点阵显示技术有了更全面的认识。